



ALPHA COLLEGE OF ENGINEERING
THIRUMAZHISAI, CHENNAI
(Approved by AICTE and Affiliated to Anna University)

PROJECT REPORT

SwiftShip Tracker

A Salesforce CRM solution with Agentforce AI for parcel booking, tracking and delivery management

Item	Details
Project Title	SwiftShip Tracker – Salesforce Implementation with Agentforce AI
Team ID	SWTID-2026-4149
Team Lead	VIJAYAKUMAR.D
Team Member	YASHWANTH.P
Team Member	BALAJI.K
Team Member	KUMARAVEL J
Team Member	MOHAMED ADHIL

Alpha College of Engineering, Thirumazhisai, Chennai

Abstract

Parcel booking and tracking is an everyday service, yet customers still depend on phone calls, scattered applications and manual registers to find out where their parcel is. This project report describes SwiftShip Tracker, a Salesforce CRM solution that our team designed and built to bring parcel booking, delivery tracking and customer communication into one platform.

The solution models the parcel lifecycle with four custom objects (Parcel, Delivery, Sender and Receiver) and the relationships between them. An auto-launched Flow looks up a parcel from its Parcel ID and passes the record to a Prompt Builder template, which produces a clear tracking update containing the parcel name, ID, status, weight and estimated delivery date. An Agentforce agent, with a dedicated Parcel Details subagent and a Flow action, lets a customer simply ask for their parcel details in a conversation. Permission sets give the agent user controlled access to the data.

The report follows the full project lifecycle: ideation with problem statements, an empathy map and brainstorming; requirement analysis with the customer journey, functional and non-functional requirements and data flow; design with the problem–solution fit and architecture; Agile planning with a product backlog, sprints, velocity and burndown; milestone-by-milestone development in the Salesforce Developer Org; testing; and a discussion of advantages, limitations and future scope.

Key facts	
Platform	Salesforce Developer Edition
AI components	Agentforce agent, subagent, Prompt Builder template
Automation	Auto-launched Flow (Parcel Details)
Data model	Parcel__c, Delivery__c, Sender__c, Receiver__c
Delivery approach	Agile, six sprints, 37 story points

Keywords: Salesforce, Agentforce, Prompt Builder, Flow, parcel tracking, CRM, custom objects, conversational AI.

Acknowledgement

We would like to express our sincere gratitude to everyone who supported us during this project. We thank our Principal, and our Head of the Department, for providing the facilities and encouragement needed to complete this work at Alpha College of Engineering.

We are deeply thankful to our project guide for the guidance, timely feedback and motivation given to us throughout the project. We also thank the faculty members and the Salesforce learning community, whose documentation and learning resources helped us understand Agentforce, Prompt Builder and Flow Builder.

Finally, we thank our families and friends for their constant support, and every member of our team for the cooperation, discipline and hard work that turned an idea on paper into a working solution.

Table of Contents

Abstract.....	1
Acknowledgement	1
1. Introduction.....	1
1.1 User Story.....	1
1.2 Project Overview	1
1.3 Purpose	1
1.4 Scope of the Project.....	1
1.5 Objectives.....	1
1.6 Stakeholders and Target Users.....	1
1.7 Organisation of this Report	1
2. Ideation Phase	1
2.1 Problem Statement.....	1
2.2 Empathy Map Canvas	1
2.3 Brainstorming and Idea Prioritization	1
3. Requirement Analysis Phase.....	1
3.1 Customer Journey Map.....	1
3.2 Solution Requirements	1
3.3 Data Flow Diagram	1
3.4 Technology Stack	1
4. Project Design Phase	1
4.1 Problem–Solution Fit	1
4.2 Proposed Solution	1
4.3 Solution Architecture	1
4.4 Data Model.....	1
5. Project Planning & Scheduling	1
5.1 Project Planning.....	1
5.2 Product Backlog and Sprint Schedule	1
5.3 Project Tracker, Velocity and Burndown	1
5.4 Team Roles and Responsibilities	1
5.5 Risks and Mitigation.....	1
6. Project Development Phase.....	1
Phase 1: Requirement Analysis & Planning	1
Phase 2: Salesforce Development – Backend & Configurations	1
Phase 3: UI/UX Development & Customization	1
Phase 4: Data Migration, Testing & Security	1
Phase 5: Deployment, Documentation & Maintenance.....	1
7. Functional & Performance Testing	1
7.1 Functional test cases.....	1
7.2 Performance observations.....	1
8. Advantages & Disadvantages.....	1
8.1 Advantages.....	1
8.2 Disadvantages	1
8.3 Our assessment	1

9. Conclusion	1
10. Future Scope	1
11. Appendix	1
11.1 Components used	1
11.2 Glossary.....	1
11.3 References.....	1

1. Introduction

1.1 User Story

The SwiftShip Tracker is designed to provide a centralized Salesforce-based parcel management solution that enables customers to book parcels, track delivery status, and manage shipments through an intuitive interface and conversational AI support. The system manages Parcel, Delivery, Sender, and Receiver information using Salesforce custom objects and relationships. Flows automate parcel operations and notifications, while Agentforce AI and Prompt Builder enable customers to retrieve parcel information and interact with the system conversationally. Delivery agents can update parcel status in real time, while Experience Cloud, security controls, reports, and dashboards provide secure customer access and operational visibility.

This single user story captures what we set out to build. Every chapter of this report traces back to it: the problem we studied, the requirements we derived, the architecture we designed, the sprints we planned and the configuration we carried out inside the Salesforce Developer Org.

1.2 Project Overview

SwiftShip Tracker is a Salesforce CRM solution developed by our team to simplify parcel booking, shipment tracking and delivery management. It supports the complete parcel lifecycle, from booking and dispatch through in-transit tracking and delivery confirmation. Customers can access shipment information on their own, while delivery agents can update parcel details and delivery status from a single, centralized platform.

We chose Salesforce as the foundation because it gives us a secure, cloud-based data layer, point-and-click automation through Flow Builder, a mature security model and, more recently, Agentforce and Prompt Builder for generative-AI features. By combining these capabilities, SwiftShip Tracker turns a traditionally support-heavy activity (“Where is my parcel?”) into a self-service conversation.

The parcel lifecycle handled by the solution is summarised below.

Stage	What happens in SwiftShip Tracker
Booking	The sender provides parcel, sender and receiver details; a Parcel record is created with an auto-generated Parcel ID.
Dispatch	The parcel is picked up from the origin and its status moves from Booked to In Transit.
In Transit	Delivery agents update the current location and estimated delivery date on the Delivery record.
Out for Delivery	The status changes to Out for Delivery and the receiver can be informed.
Delivered	Delivery is confirmed, the status is closed and the data is available for reporting.
Support / Tracking	Customers ask the Agentforce agent for the status of a parcel by giving the Parcel ID.

1.3 Purpose

The purpose of SwiftShip Tracker is to:

- Centralize parcel and delivery information in one Salesforce org.
- Simplify parcel booking and tracking for customers and staff.
- Provide real-time shipment visibility to every stakeholder.
- Automate delivery updates and customer notifications.
- Enable conversational parcel tracking using Agentforce AI and Prompt Builder.
- Improve agent efficiency through Salesforce Flows and dashboards.
- Provide secure, role-based access to customers, agents, support staff and administrators.

1.4 Scope of the Project

Because this project was carried out in a Salesforce Developer Edition org for academic purposes, we defined the scope clearly at the start so that the work stayed focused and achievable within the sprint schedule.

In scope	Out of scope (future work)
Custom objects Parcel, Delivery, Sender and Receiver with relationships	Production deployment, UAT and sandbox pipelines
Custom tabs and Lightning App navigation	CI/CD automation for Agentforce deployment
Prompt Builder template that retrieves parcel details by Parcel ID	Live integration with third-party courier or carrier APIs
Auto-launched Flow that fetches the parcel and calls the prompt template	Real SMS gateway and map-based live tracking
Agentforce agent with a Parcel Details subagent and Flow action	Large-scale performance and load testing
Permission sets and role-based access for the agent user	Full customer self-service portal rollout at scale

1.5 Objectives

- **Streamline parcel operations** by automating booking, tracking and delivery notifications to reduce manual updates.
- **Improve customer satisfaction** with transparent, real-time shipment status and self-service tracking options.
- **Enhance agent efficiency** through centralized dashboards and guided Flows for quick status updates.
- **Enable business insights** with reports and dashboards showing parcel trends, delivery performance and agent activity.
- **Ensure security and reliability** by implementing Salesforce role hierarchies, sharing rules and audit-based data access.

1.6 Stakeholders and Target Users

Stakeholder	Role in the system
Customer / Sender	Books parcels, tracks deliveries and receives real-time updates through the portal or the AI agent.
Delivery Agent	Manages assigned parcels, updates delivery status and location, and ensures timely handover.
Customer Support Team	Handles delivery-related issues, responds to customer queries and keeps communication smooth.
Admin Team	Configures the Salesforce environment, manages users, roles and security, and monitors performance.
System Integrator / Developer	Implements automation, sets up the AI agent and builds backend integrations.

1.7 Organisation of this Report

The report follows the lifecycle we used while working on the project:

Chapter	Content
1 – Introduction	User story, overview, purpose, scope, objectives and stakeholders.
2 – Ideation Phase	Problem statements, empathy map and brainstorming with idea prioritization.
3 – Requirement Analysis	Customer journey map, functional and non-functional requirements, data flow and technology stack.
4 – Project Design	Problem–solution fit, proposed solution, architecture and data model.
5 – Planning & Scheduling	Agile planning, product backlog, sprints, velocity and burndown.
6 – Project Development	Five development phases and the milestone-by-milestone implementation in Salesforce.
7 – Testing	Functional and performance testing of the flow, prompt and agent.
8 to 11	Advantages and disadvantages, conclusion, future scope and appendix.

2. Ideation Phase

The ideation phase is where we stepped away from the technology and focused on people. Before touching Salesforce, we asked who would use the system, what frustrates them today and which of the available options would solve those frustrations best. We used three well-known design-thinking tools: customer problem statements, an empathy map canvas and a brainstorming and prioritization exercise.

2.1 Problem Statement

We captured the pain points of each user group using the Customer Problem Statement template: “I am (user), I’m trying to..., but..., because..., which makes me feel...”. Writing the problems this way forced us to describe the situation from the user’s point of view, including how it makes them feel.

PS	I am (Customer)	I’m trying to	But	Because	Which makes me feel
PS-1	Customer / Sender	book a parcel easily	booking requires support assistance or multiple applications	parcel operations are not centralized	frustrated
PS-2	Customer / Sender	track my parcel	I do not have a simple real-time tracking experience	shipment information is difficult to access	confused
PS-3	Delivery Agent	update parcel status	status updates require manual handling	there is a need for centralized operational tracking	concerned
PS-4	Customer	receive delivery updates	notifications may not be available at every delivery stage	communication needs to be automated	uncertain
PS-5	Admin	monitor parcel operations	parcel flow and delivery performance are difficult to monitor	operational data needs centralized reporting	concerned

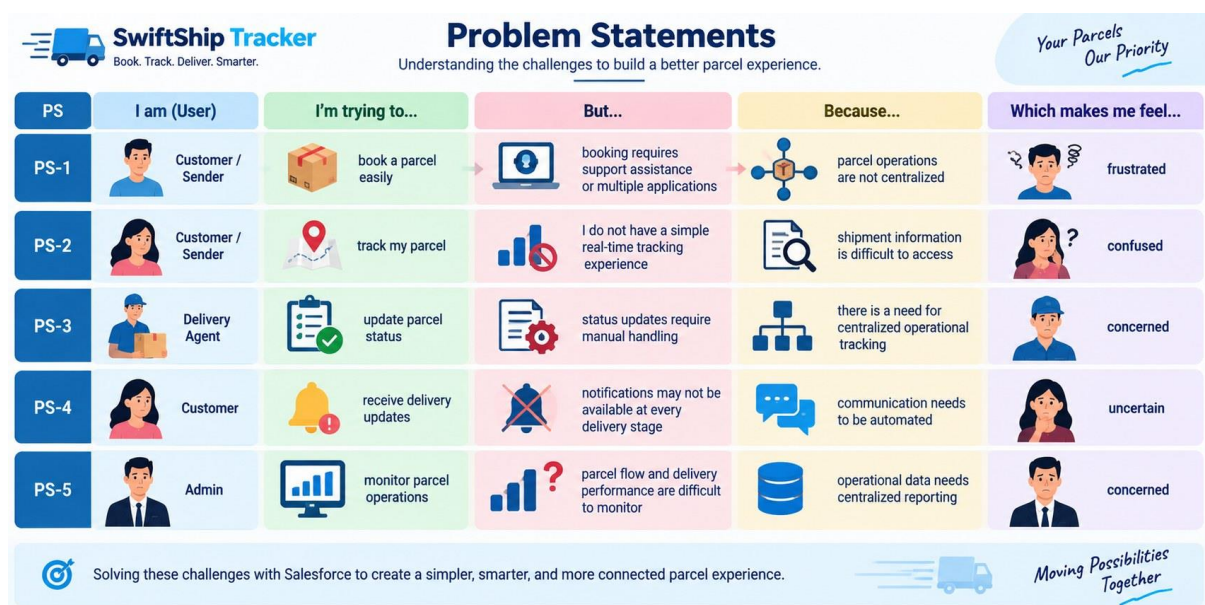


Figure 1: Problem statements of the SwiftShip Tracker users

What the problem statements tell us

- **Booking (PS-1):** customers are forced to depend on support staff or juggle several applications. The remedy is one centralized booking interface.
- **Tracking (PS-2):** shipment information exists but is hard to reach. A simple, self-service view of the current status is required.
- **Agent updates (PS-3):** manual status updates are slow and error-prone, so agents need a guided, automated way to record progress.
- **Notifications (PS-4):** customers feel uncertain when updates are missing at any delivery stage, so communication must be automated.
- **Monitoring (PS-5):** administrators cannot judge delivery performance without centralized reports and dashboards.

2.2 Empathy Map Canvas

To understand the main user in more depth, we built an empathy map for the **Customer / Sender** persona. The canvas helped us move beyond features and think about what this person actually sees, hears, thinks and feels during the parcel journey.

Quadrant	Observations for the Customer / Sender
Think & Feel	Wants a simple, convenient and reliable way to book and track parcels without repeatedly contacting support. Hopes the parcel reaches on time and safely, feels more confident with real-time updates, and gets concerned when there is no tracking information.
Hear	Receives delivery confirmations and status updates from the parcel service, information about delivery timelines, feedback from friends, family or colleagues, and announcements about service changes.
See	Parcel details, tracking status, estimated delivery date and delivery information, a user-friendly portal or mobile interface, and notifications or e-mails about shipment progress.
Say & Do	Books parcels through the portal or app, checks shipment status regularly, asks for delivery updates when needed, provides feedback and shares tracking information with recipients.
Pain	Delays in delivery, confusion due to lack of clear tracking information, difficulty accessing shipment details, dependence on customer support or multiple apps, and uncertainty about delivery timelines.
Gain	Easy and convenient parcel booking, real-time tracking and delivery updates, timely notifications at every stage, better visibility of delivery progress and peace of mind that the parcel is on track.

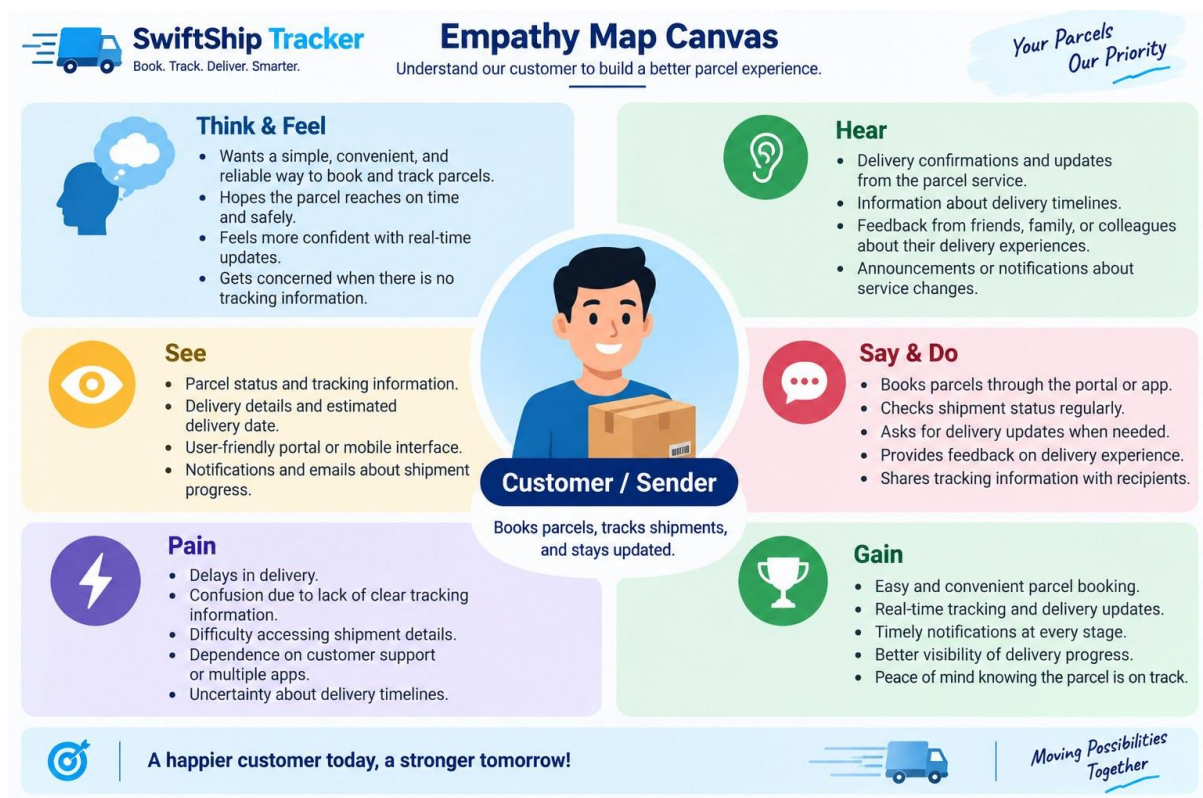


Figure 2: Empathy map canvas for the Customer / Sender persona

Key insight: the customer's biggest pain is uncertainty, and the biggest gain is visibility. This directly shaped the product: tracking had to be self-service and available at any moment, which is why a conversational Agentforce agent became the centrepiece of the solution.

2.3 Brainstorming and Idea Prioritization

We began by listing every reasonable way of managing parcels, then compared them against what the empathy map told us customers need.

Ideas considered

Idea	Description	Main limitation
Manual registers	Parcel and delivery details are written and maintained by hand.	Time-consuming and prone to errors.
Spreadsheet tracking	Parcel bookings, status and delivery information are kept in spreadsheets.	Limited collaboration and scalability.
Standalone parcel portal	A separate parcel booking and tracking portal is developed from scratch.	Requires additional development, maintenance and infrastructure.
Salesforce CRM with Agentforce AI	Centralized parcel management, tracking, automation and AI-based customer assistance.	Needs careful configuration, but is scalable and future-ready.

Comparison of the options

Criterion	Manual registers	Spreadsheets	Standalone portal	Salesforce + Agentforce
Centralized data	Low	Medium	High	High
Automation	None	Limited	Needs custom code	Built in (Flows)
Real-time visibility	Low	Low	Medium	High
Conversational AI	No	No	Extra effort	Yes (Agentforce)
Scalability	Low	Low	Medium	High
Build and upkeep effort	Low	Low	High	Medium

Final selection

After comparing the options, we selected **Salesforce CRM with Agentforce AI**. The decision rests on three strengths that matter most for the problems we identified:

- **Automation:** Salesforce Flows and automated notifications reduce manual updates.
- **Scalability:** the platform supports growing parcel, delivery, customer and agent operations.
- **Real-time visibility:** parcel status and delivery information are available through Salesforce and Agentforce AI.

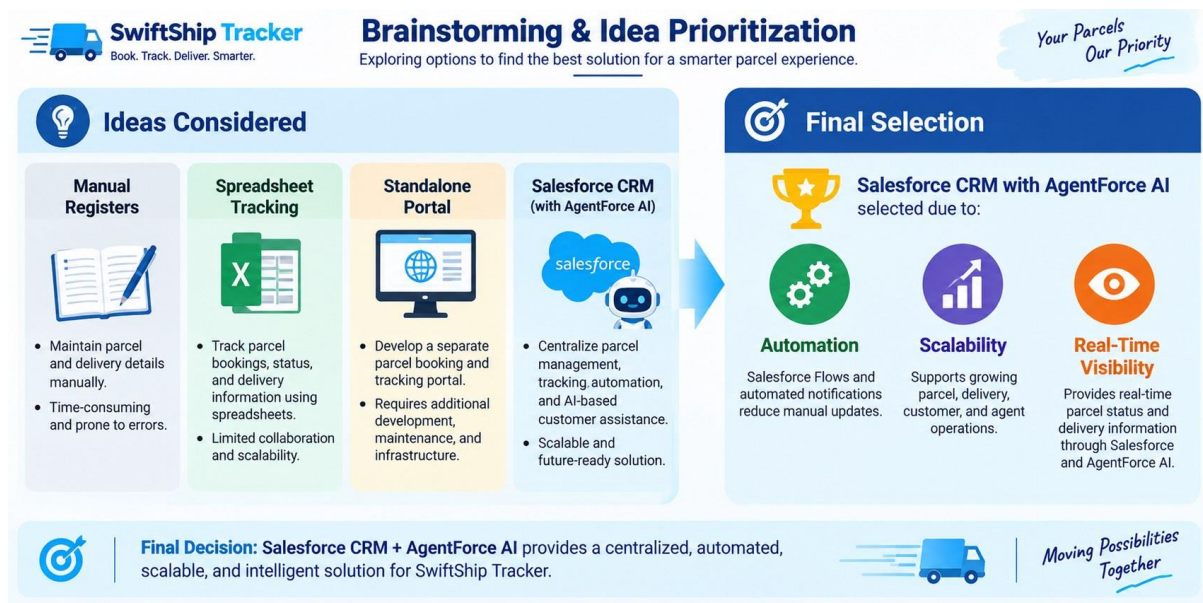


Figure 3: Brainstorming and idea prioritization

Beyond these three, Salesforce also gave us centralized parcel management, conversational AI assistance, secure role-based access, and reports and dashboards, all within one platform. This is why the final decision was a centralized, automated, scalable and intelligent solution for SwiftShip Tracker.

3. Requirement Analysis Phase

With the problem and the chosen approach clear, the requirement analysis phase translated user needs into concrete, testable requirements. We mapped the customer journey, listed the functional and non-functional requirements, drew the data flow and fixed the technology stack.

3.1 Customer Journey Map

The customer journey map follows a customer from the first enquiry to the final reporting stage. It identifies the touchpoints where SwiftShip Tracker must perform and the emotion we want the customer to feel at each step. The journey consists of the following stages: Parcel Booking, Parcel Record Creation, Dispatch & Shipment, In-Transit Tracking, Out for Delivery, Delivery Confirmation, Customer Notification, and Feedback / Support.



Figure 4: Customer journey map – from booking to delivery

#	Stage	What happens	Customer thought	Emotion
1	Parcel Inquiry	Customer checks parcel details, pricing and delivery options.	"What are the delivery options and cost?"	Curious
2	Parcel Booking	Customer books a parcel by providing sender, receiver and parcel details.	"That was easy to book!"	Happy
3	Record Creation	System creates a parcel record in Salesforce.	"My parcel is now registered."	Confident
4	Dispatch	Parcel is picked up and dispatched from the origin location.	"Great, my parcel is dispatched."	Satisfied
5	In-Transit Tracking	Parcel is in transit and can be tracked in real time.	"I can track my parcel in real time."	Reassured
6	Out for Delivery	Parcel is out for delivery to the receiver's location.	"It's out for delivery now."	Anticipated
7	Delivery Confirmation	Parcel is delivered successfully.	"My parcel is delivered!"	Delighted
8	Customer Notification	Customer receives notifications at every stage.	"I got a notification."	Informed
9	Reporting	Data is captured for reporting and performance analysis.	"I can see the summary and reports."	Satisfied

Reading the journey from left to right, the emotion improves steadily as uncertainty is replaced by information. The only stage that needs no customer action at all, Reporting, is aimed at the business, which closes the loop between operations and decision-making.

3.2 Solution Requirements

3.2.1 Functional requirements

FR No	Requirement	How it is addressed
FR-1	Parcel booking and record creation	Parcel__c object with auto-numbered Parcel ID
FR-2	Parcel status tracking	Status picklist: Booked, In Transit, Out for Delivery, Delivered
FR-3	Weight management	Weight number field on Parcel__c
FR-4	Estimated delivery date management	Estimated Delivery Date field on Parcel__c and Delivery__c
FR-5	Sender information management	Sender__c object (address, contact, e-mail)
FR-6	Receiver information management	Receiver__c object (address, contact, e-mail)
FR-7	Delivery and current-location tracking	Delivery__c object with a Geolocation field
FR-8	Agent parcel status updates	Lightning record pages and Flows for agents
FR-9	Automated customer notifications	Email alerts and templates triggered by Flows
FR-10	AI-powered parcel tracking	Agentforce agent, Prompt Builder template and Flow action
FR-11	Reports and dashboards	Salesforce reports and dashboards
FR-12	Customer access through Experience Cloud	Experience Cloud site with read-only customer access

3.2.2 Non-functional requirements

NFR	Description
Usability	Intuitive Salesforce and customer-facing interface.
Security	Role-based and field-level access control.
Reliability	Accurate parcel status and delivery information.
Performance	Timely Flow and Agentforce responses.
Availability	Salesforce cloud-based access.
Scalability	Supports growing parcel and delivery operations.
Data Integrity	Controlled access and relationship-based parcel data.
Maintainability	Flows, prompts, topics and configurations can be updated easily.

3.2.3 User roles and access

Different users need different levels of access to the same data. The access we planned for each role is summarised below and is explained in detail in the security model in Chapter 6.

Role	Parcel	Delivery	Sender / Receiver	Other access
Administrator	Full	Full	Full	All objects, setup and security
Delivery Agent	Read / Edit	Read / Edit	Read / Edit	Update Delivery Status permission set
Customer	Read own	Read own	Read own	Experience Cloud, read-only
Support Staff	Read	Read	Read	Cases for issue resolution

3.3 Data Flow Diagram

The data flow diagram shows how information moves through the system, from the moment a customer books a parcel to the moment management reads a report. Two flows matter: the end-to-end parcel flow and the AI tracking flow.

End-to-end parcel flow

```
Customer / Sender
  ↓ Parcel Booking
Parcel Record
  ↓
Sender + Receiver Information
  ↓
Delivery Record
  ↓
Parcel Status Update
  ↓
Current Location & Estimated Delivery
  ↓
Automated Notifications
  ↓
Customer Tracking / Agentforce AI
  ↓
Reports & Dashboards
```

AI tracking flow

```
Customer Query
  ↓
Agentforce AI
  ↓
Prompt Builder - Retrieve Parcel Details
  ↓
Parcel Record
  ↓
Status / Weight / Estimated Delivery
  ↓
Conversational Response
```

The Prompt Builder implementation retrieves Parcel ID, Status, Weight and Estimated Delivery Date information for tracking responses.

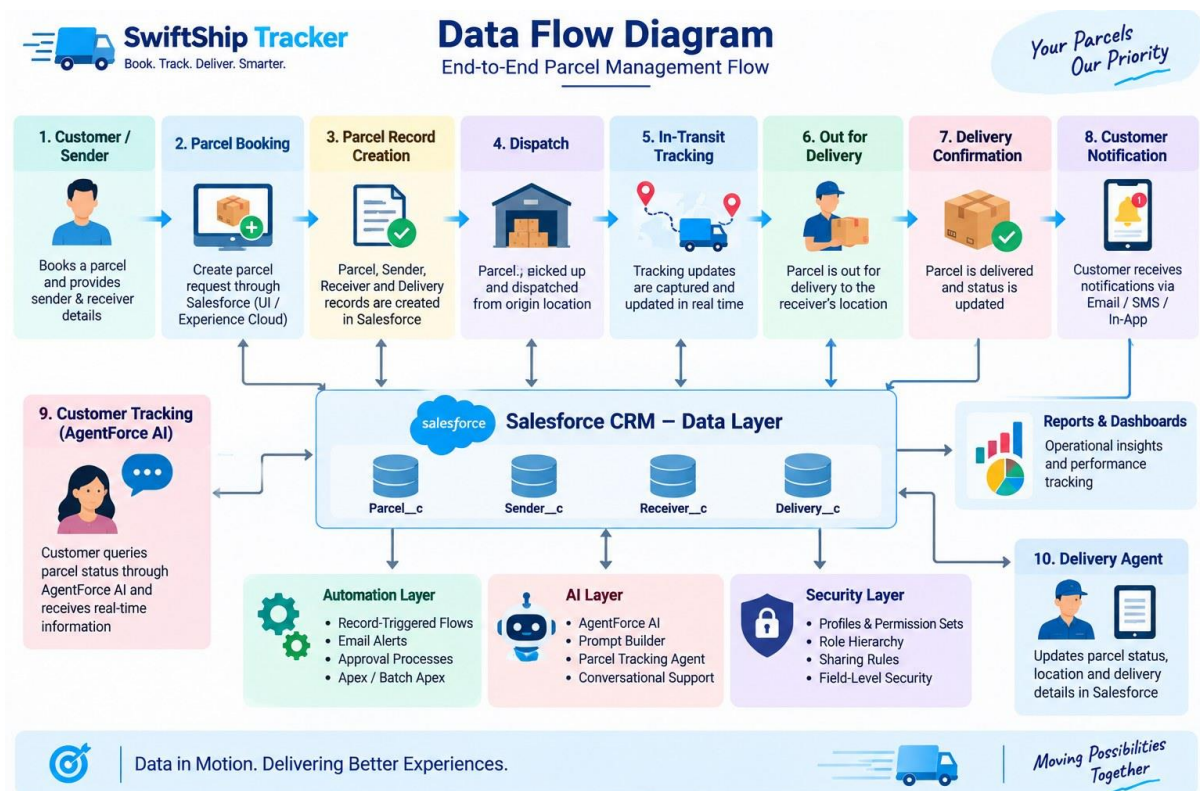


Figure 5: Data flow diagram – end-to-end parcel management flow

Step	Activity	Data involved
1	Customer / sender books a parcel and provides sender and receiver details	Parcel, sender and receiver inputs
2	Parcel booking request is created through Salesforce (UI / Experience Cloud)	Booking request
3	Parcel, Sender, Receiver and Delivery records are created in Salesforce	Parcel__c, Sender__c, Receiver__c, Delivery__c
4	Parcel is picked up and dispatched from the origin location	Status = In Transit
5	Tracking updates are captured and updated in real time	Current location, estimated delivery
6	Parcel is out for delivery to the receiver's location	Status = Out for Delivery
7	Parcel is delivered and the status is updated	Status = Delivered
8	Customer receives notifications via e-mail / SMS / in-app	Notification content
9	Customer queries parcel status through the Agentforce AI agent	Parcel ID, prompt response
10	Delivery agent updates parcel status, location and delivery details	Parcel and delivery fields

At the centre of the diagram sits the Salesforce CRM data layer. Around it are three supporting layers: the automation layer (record-triggered flows, e-mail alerts, approval processes, Apex and Batch Apex), the AI layer (Agentforce AI, Prompt Builder, parcel tracking agent, conversational support) and the

security layer (profiles and permission sets, role hierarchy, sharing rules and field-level security). Reports and dashboards read from the same data layer to provide operational insights.

3.4 Technology Stack

Layer	Technology
Platform	Salesforce Developer Edition
UI	Lightning App Page, Lightning Record Pages, Experience Cloud
Database	Salesforce Custom & Standard Objects
Business Logic	Salesforce Flows, Apex Classes
Automation	Record-Triggered Flows, Auto-Launched Flows, Scheduled Flows
AI	Agentforce AI, Prompt Builder
Notifications	Email Alerts & Email Templates
Processing	Batch Apex
Security	Profiles, Permission Sets, Role Hierarchy, Field-Level Security, Sharing Rules
Reporting	Reports & Dashboards

Why this stack? Every layer is native to Salesforce, so there is no external server to host, no separate database to secure and no integration glue to maintain. Flow Builder keeps the business logic visible to non-programmers; Prompt Builder lets us shape AI responses with merge fields from real records; and Agentforce wires the prompt into a conversational agent that respects the org's permissions.

4. Project Design Phase

In the design phase we mapped each problem to a Salesforce capability, described the proposed solution, drew the overall architecture and finalised the data model that the development phase would implement.

4.1 Problem–Solution Fit

The problem–solution fit checks that every pain point found during ideation has a matching feature and a measurable benefit. If a problem has no solution, the product has a gap; if a feature has no problem, it is wasted effort.

Problem	Solution	Benefit
Parcel booking is difficult	Salesforce parcel management	Centralized booking
Tracking requires support	Agentforce AI + Prompt Builder	Self-service tracking
Manual status updates	Salesforce Flows	Automated processes
Delivery communication gaps	Email alerts	Timely notifications
Scattered shipment information	Parcel, Delivery, Sender & Receiver objects	Structured data
Difficult operational monitoring	Reports & dashboards	Better visibility
Different users need different access	Profiles, roles & permission sets	Secure access

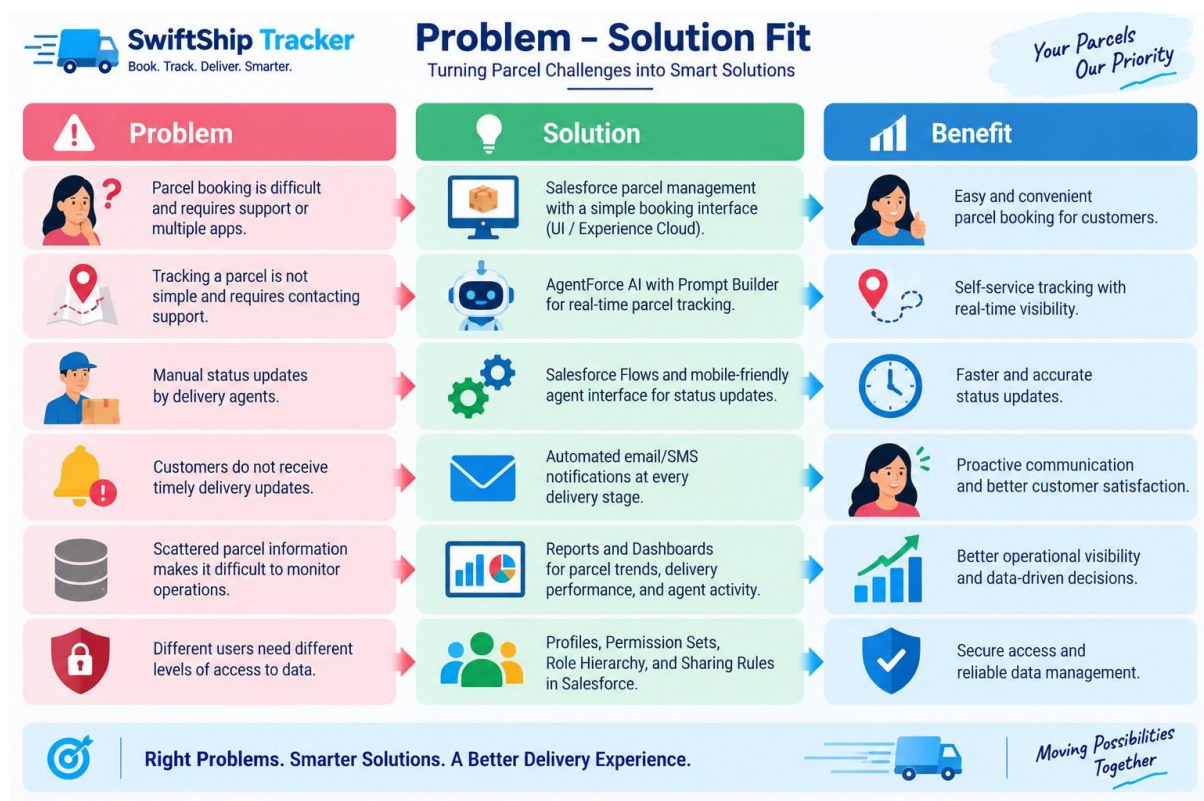


Figure 6: Problem–solution fit for SwiftShip Tracker

4.2 Proposed Solution

Parameter	Description
Problem	Difficult and inefficient parcel booking and tracking.
Solution	Salesforce-based parcel management system.
Innovation	Agentforce AI + Prompt Builder for conversational self-service tracking and automated Salesforce Flows.
Impact	Centralized booking, real-time tracking visibility, timely notifications and efficient delivery management.
Business Model	Parcel, customer, delivery-agent, support and administrator operations.
Scalability	Supports growing parcel operations, Experience Cloud and integration with external logistics systems.

4.3 Solution Architecture

SwiftShip Tracker follows a layered architecture on the Salesforce platform. Users reach the system through a web portal (Experience Cloud), the Salesforce mobile app, the Lightning UI or the conversational AI agent. Behind these channels, four layers cooperate:

Layer	Components
Users	Customer, Delivery Agent, Support Staff, Admin
Data Layer	Parcel, Delivery, Sender, Receiver
Automation Layer	Flows, Approval Processes, Apex Classes, Batch Apex, Email Alerts
AI Layer	Agentforce AI, Prompt Builder, Parcel Tracking Agent
Security Layer	Profiles, Roles, Permission Sets, Sharing Rules, Field-Level Security
Output	Reports & Dashboards, Parcel Tracking & Operational Insights

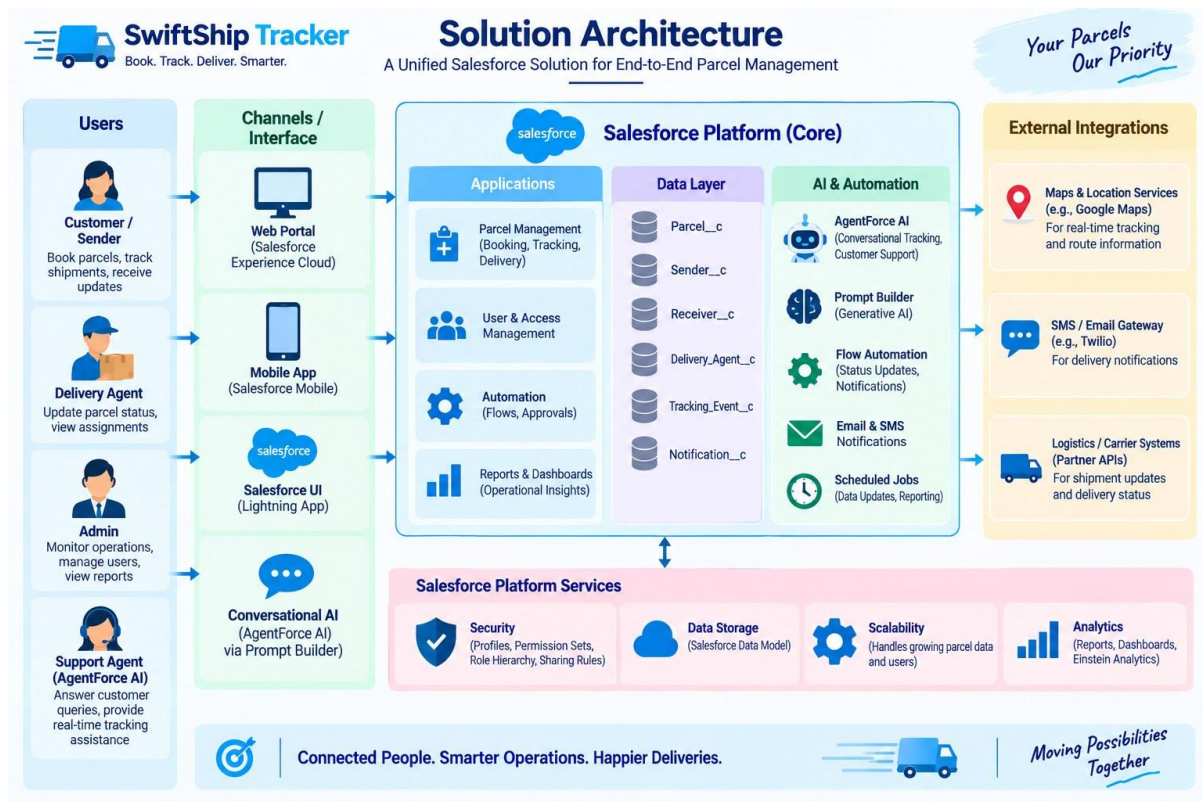


Figure 7: Solution architecture – a unified Salesforce solution for end-to-end parcel management

The architecture diagram also shows optional external integrations: map and location services for route information, an SMS or e-mail gateway for delivery notifications, and logistics or carrier systems for shipment updates. These are part of the scalability plan and are listed in the future scope; the implemented prototype relies entirely on the Salesforce platform services for security, data storage, scalability and analytics.

4.4 Data Model

The data model is built around four custom objects. Parcel__c is the heart of the model; Sender__c and Receiver__c describe the two ends of a shipment and Delivery__c records where the parcel currently is.

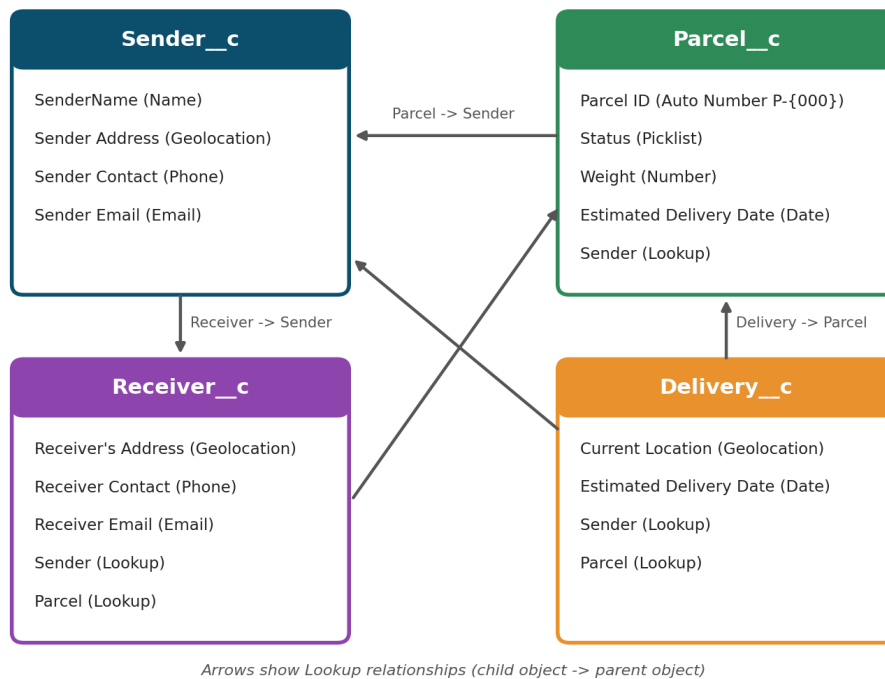


Figure 8: Data model of SwiftShip Tracker (custom objects and lookups)

Object	Purpose	Key relationships
Parcel__c	Stores core parcel details such as Parcel ID, Status, Weight and Estimated Delivery Date.	Lookup to Sender__c; referenced by Delivery__c and Receiver__c.
Delivery__c	Stores delivery-specific information such as current location and estimated delivery date.	Lookups to Sender__c and Parcel__c.
Sender__c	Captures sender address, contact number and e-mail for confirmations and updates.	Linked to many parcels, so sender history can be traced.
Receiver__c	Stores receiver address, contact and e-mail for delivery confirmation.	Lookups to Sender__c and Parcel__c.

5. Project Planning & Scheduling

5.1 Project Planning

We managed the project with the Agile methodology. Work was broken down into epics (the functional areas), user stories (what a user needs) and story points (a relative estimate of effort). The work was then executed in short, time-boxed sprints, with the team's velocity used to judge how much could be completed in each sprint.

- **Agile methodology** – small increments, regular review and quick course correction.
- **Epics → Stories → Story Points** – each epic is divided into user stories that are sized in points.
- **Sprint-based execution** – six sprints of six working days each.
- **Velocity calculation** – used for estimation and for checking progress against the plan.

5.2 Product Backlog and Sprint Schedule

Sprint	Epic	USN	User story / task	Pts	Priority	Member
1	Developer Setup	USN-1	As a system administrator, I want to create and configure the Salesforce environment so that I can build the SwiftShip Tracker parcel management system.	3	High	Member 1
2	Data Modeling	USN-2	As an admin, I want to create Parcel, Delivery, Sender and Receiver objects with proper relationships so that parcel and delivery data is stored and managed systematically.	5	High	Member 2
2	Data Modeling	USN-3	As a staff member, I want dedicated tabs and a Lightning App for SwiftShip modules so that I can easily navigate and manage parcel and delivery operations.	5	High	Member 3
3	Automation	USN-4	As a system, I want to validate parcel details, mandatory fields and booking inputs so that invalid or incomplete parcel records are prevented.	3	High	Member 4
3	Automation	USN-5	As a delivery coordinator, I want automated Flows to update parcel status and delivery information so that parcel processing requires minimal manual effort.	3	High	Member 4
4	Apex	USN-6	As a system, I want Apex triggers and classes to automatically perform critical parcel and delivery actions so that important processes are handled accurately.	5	High	Member 1
4	Apex	USN-7	As an operations manager, I want Batch Apex jobs to process and update parcel or delivery records automatically so that operational data remains accurate and up to date.	5	High	Member 2
5	Reports	USN-8	As a manager, I want reports showing parcel status, delivery performance and operational metrics so that I can monitor SwiftShip operations and make informed decisions.	4	High	Member 3
6	Dashboards & AI	USN-9	As a manager, I want dashboards showing parcel status, delivery performance and key metrics, while Agentforce provides AI-powered tracking assistance, so that I can monitor operations efficiently.	4	Medium	Member 4

Note on team members

Replace “Member 1” to “Member 4” with the actual names of your team members.

5.3 Project Tracker, Velocity and Burndown

Sprint tracker

The tracker below lists each sprint with the story points planned from the product backlog. Each sprint was timed at six days, and the dates are to be filled in with the actual calendar dates followed by the team.

Sprint	Story points	Duration	Start date	Planned end date	Points completed	Release date
Sprint-1	3	6 days	[date]	[date]	3	[date]
Sprint-2	10	6 days	[date]	[date]	10	[date]
Sprint-3	6	6 days	[date]	[date]	6	[date]
Sprint-4	10	6 days	[date]	[date]	10	[date]
Sprint-5	4	6 days	[date]	[date]	4	[date]
Sprint-6	4	6 days	[date]	[date]	4	[date]
Total	37	36 days			37	

Velocity

Velocity measures how many story points the team completes per unit of time. With 37 story points delivered over six sprints of six days each:

Measure	Calculation	Result
Total story points	$3 + 10 + 6 + 10 + 4 + 4$	37 points
Average velocity per sprint	$37 \text{ points} \div 6 \text{ sprints}$	$\approx 6.2 \text{ points per sprint}$
Average velocity per day	$37 \text{ points} \div 36 \text{ days}$	$\approx 1.03 \text{ points per day}$

Burndown chart

A burndown chart plots the work remaining against time. The chart below shows the planned burndown of the 37 story points against an ideal even pace. The steeper drops in Sprints 2 and 4 reflect the heavier data-modeling and Apex sprints.

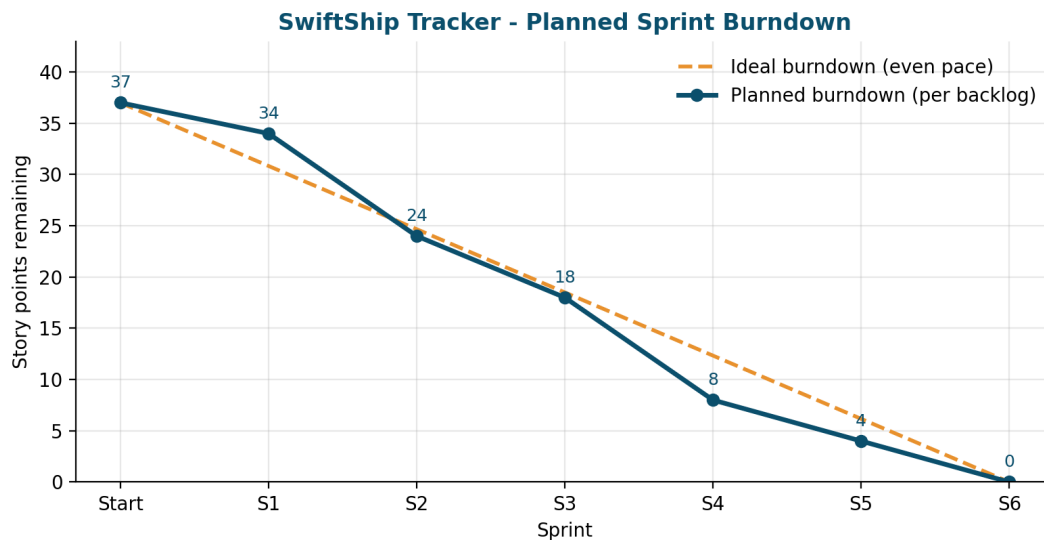


Figure 9: Planned sprint burndown chart

5.4 Team Roles and Responsibilities

Team member	User stories	Responsibility
Member 1	USN-1, USN-6	Salesforce environment setup and Apex triggers and classes
Member 2	USN-2, USN-7	Custom objects and relationships; Batch Apex jobs
Member 3	USN-3, USN-8	Tabs and Lightning App; reports
Member 4	USN-4, USN-5, USN-9	Validation, Flow automation, dashboards and Agentforce AI

5.5 Risks and Mitigation

Risk	Impact	How we handled it
Developer Edition limits (no sandbox, limited storage)	No production-like testing environment	Tested thoroughly inside the Developer Org and documented the limitation.
Incorrect or incomplete parcel data	Wrong AI answers for tracking queries	Made key fields visible in permission sets and tested with known parcel IDs.
Agent cannot read parcel data	Agent returns an error or empty response	Assigned the SwiftShip permission set to the Einstein Service Agent user.
Prompt returns wrong fields	Misleading tracking summary	Mapped each Parcel__c field as a resource and previewed the resolved prompt.
Tight sprint durations	Delay in later sprints	Prioritised core features (objects, flow, agent) in the earliest sprints.

6. Project Development Phase

The development of SwiftShip Tracker was carried out in five phases. The first phase fixed what we would build and how it would be secured; the second built the backend in Salesforce; the third prepared access control and the Agentforce agent; the fourth validated everything; and the fifth documented how the solution would be deployed and maintained.

Phase	Title	Outcome
1	Requirement Analysis & Planning	Scope, users, features, data model and security model defined.
2	Salesforce Development – Backend & Configurations	Developer org, objects, tabs, fields, Prompt Builder template and Flow.
3	UI/UX Development & Customization	Permission sets, Agentforce org setup, agent, subagent and actions.
4	Data Migration, Testing & Security	Agent, topic, flow and data validated end to end.
5	Deployment, Documentation & Maintenance	Deployment scope, maintenance, monitoring and troubleshooting plan.

Phase 1: Requirement Analysis & Planning

Objective

“Customers often face delays and confusion while booking, tracking, and managing parcel deliveries because they need to call support or use multiple apps. A conversational AI agent can simplify this by allowing customers to book parcels, get live tracking, and manage deliveries in one place.”

Key requirements

- Requirement gathering from end users and delivery agents.
- Defining system scope and user personas (Customer, Agent, Admin).
- Designing the data model with relationships between Parcel, Sender, Receiver and Delivery.
- Identifying use cases such as parcel booking, tracking and delivery update automation.
- Selecting tools: Salesforce Flows, Prompt Builder and Agentforce AI.

Defining project scope and objectives

Project scope

1. Build a scalable CRM solution for parcel delivery businesses to manage customer bookings, shipments and delivery operations efficiently.
2. Support the end-to-end parcel lifecycle: parcel booking → dispatch → tracking → delivery confirmation.
3. Include comprehensive data model design, automation (Flows, Apex, Email Alerts), an AI-powered prompt builder and reporting dashboards for effective management.
4. Provide role-based access for admins, delivery agents and customers through Salesforce profiles, permission sets and Experience Cloud.

-
5. Ensure real-time parcel visibility for customers and simplified operational tracking for agents through AI-integrated conversational support.

Objectives summary

- Streamline parcel operations by automating booking, tracking and delivery notifications to reduce manual updates.
- Improve customer satisfaction with transparent, real-time shipment status and self-service tracking options.
- Enhance agent efficiency through centralized dashboards and guided Flows for quick status updates.
- Enable business insights with reports and dashboards showing parcel trends, delivery performance and agent activity.
- Ensure security and reliability by implementing Salesforce role hierarchies, sharing rules and audit-based data access.

Gathering and analysing user needs

Users involved

- **Customers / Senders** – book parcels, track deliveries and receive real-time updates through the portal or AI agent.
- **Delivery Agents** – manage assigned parcels, update delivery status and ensure timely handover to customers.
- **Customer Support Team** – handle delivery-related issues, respond to customer queries and ensure smooth communication.
- **Admin Team** – configure the Salesforce environment, manage user roles and security settings and monitor operational performance.
- **System Integrator / Developer** – implement automation, AI agent setup and backend integrations.

Key functional needs

- Seamless parcel booking and tracking interface for customers through a Salesforce Experience site or agent assistance.
- Ability for agents to update parcel status (Booked → In Transit → Out for Delivery → Delivered) in real time.
- Automated notifications and e-mail alerts to customers on every delivery stage update.
- Support for Sender and Receiver details management with address, contact and e-mail fields for communication.
- AI agent integration (via Prompt Builder) for real-time parcel status queries and automated assistance.
- Reports and dashboards to monitor parcel flow, delivery performance and agent activity for admins.
- Security controls with role-based access, profiles and permission sets for different user levels.

Tools used

Tool	Used for
Salesforce Developer Org	Configuration, development and testing.
Prompt Builder (Agentforce AI)	Creating intelligent parcel-tracking prompts and responses.
Salesforce Flow Builder	Automating booking, tracking and delivery updates.
Google Sheets / Docs	Initial data planning, requirement documentation and system mapping.

Identifying key Salesforce features and tools

Category	Features planned
Custom objects	Parcel__c, Delivery__c, Sender__c, Receiver__c
Standard objects	Account, Contact, Case, Task, EmailMessage, User
Automations	Record-Triggered Flows, Auto-Launched Flows, Approval Processes, Scheduled Flows
Apex	Apex classes for automated parcel operations (for example lead-to-parcel mapping and delivery notifications) and Batch Apex for periodic updates or notifications
UI	Lightning App Page (SwiftShip Tracker App), Dynamic Forms for parcel details, Lightning Record Pages and Experience Cloud for customer access
Agentforce AI & Prompt Builder	AI agent trained to fetch parcel data based on Parcel ID and respond to tracking or delivery status queries
Email services	Custom email alerts and templates for booking confirmations, delivery updates and completion acknowledgments
Security	Profiles, Permission Sets, Role Hierarchies, Field-Level Security and Sharing Rules for Admin, Agent and Customer roles

Designing the data model and security model

Data model

- **Parcel__c** stores core parcel details such as Parcel ID, Status, Weight and Estimated Delivery Date. It is linked to Sender__c and Receiver__c to fetch their details and is connected with Delivery__c to track real-time parcel location and status.
- **Delivery__c** stores delivery-specific information such as current location, estimated delivery date and the linked parcel. It is connected with Sender__c and Parcel__c and is used by agents to update and monitor delivery status during the parcel lifecycle.
- **Sender__c** captures sender address, contact number and e-mail. It is linked to multiple parcels to track sender history and associated deliveries, and is used for booking confirmations and delivery updates via automated e-mails.
- **Receiver__c** stores receiver address, contact and e-mail for communication and delivery confirmation. It is linked to Sender__c and Parcel__c to associate both ends of the shipment and is used by agents for delivery reference and automated e-mail notifications.

Security model

Security element	Design
Role hierarchy	Administrator → Delivery Manager → Agent → Customer
Admin profile	Full access to all objects and records.
Delivery Agent profile	Read / Edit access to Parcel, Delivery, Sender and Receiver objects.
Customer profile	Read-only access to their own parcel details via Experience Cloud.
Support Staff profile	Access to Cases and Parcel details for issue resolution.
Permission sets	Grant additional privileges, for example “Update Delivery Status” and “Access Agent Console”.
Record-level security	Sharing rules on Parcel__c and Delivery__c give controlled access to agents and managers; customers can only view records linked to their own Sender or Receiver details.
Field-level security	Sensitive fields such as Sender Email, Receiver Contact and Location Coordinates are restricted to Admin and Agent profiles only.

Phase 2: Salesforce Development – Backend & Configurations

This phase focused on developing the core logic of SwiftShip Tracker. Custom objects were created for Parcel, Sender, Receiver and Delivery with relationships. Automations were built using Flows for parcel status updates, and a Prompt Builder template was prepared so that the AI agent could retrieve parcel details. Each milestone below describes what we did, the exact steps we followed and the screenshot of the result.

Milestone 1 – Salesforce Account

Introduction

Are you new to Salesforce? Not sure exactly what it is, or how to use it? Don't know where you should start on your learning journey? If you've answered yes to any of these questions, then you're in the right place. This module is for you.

Welcome to Salesforce! Salesforce is game-changing technology, with a host of productivity-boosting features, that will help you sell smarter and faster. As you work toward your badge for this module, we'll take you through these features and answer the question, "What is Salesforce, anyway?"

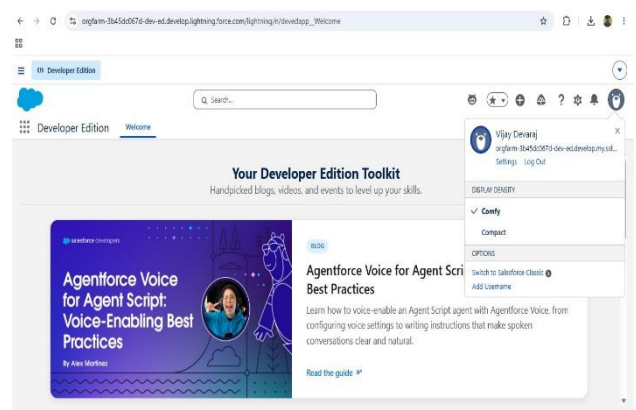
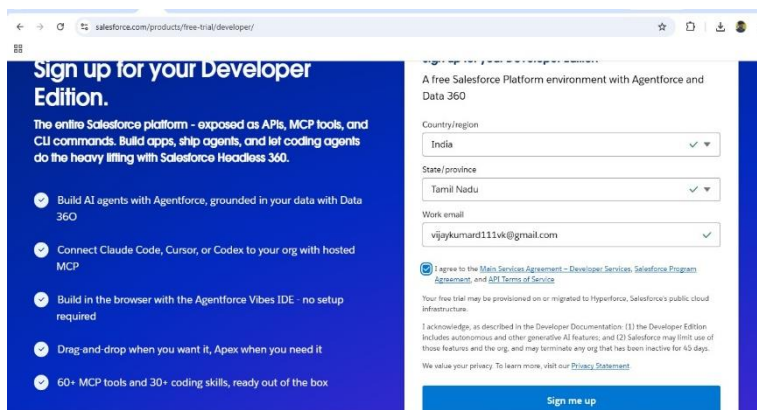
What is Salesforce?

Salesforce is your customer success platform, designed to help you sell, service, market, analyse and connect with your customers.

Creating a developer account

We created a Developer Edition org, which is free and comes with Agentforce and Data Cloud. All further configuration of SwiftShip Tracker was done in this org.

1. Go to <https://developer.salesforce.com/signup>.
2. On the sign-up form, enter the following details: first name and last name, e-mail, job title (Developer), company (college name) and country (India).
3. Click **Sign Me Up** after filling in these details.
4. Go to the inbox of the e-mail you used while signing up and click **Verify Account** to activate the account. The e-mail may take 10 to 30 minutes and sometimes up to 2 hours.
5. Give a password, answer a security question and click **Change Password**.
6. You are redirected to your Salesforce setup page.



Milestone 2 – Objects Creation

Objects are the tables of Salesforce. We created four custom objects: Parcel, Delivery, Sender and Receiver. All four were created in the same way, with reports, search and field history tracking switched on so that they can be reported on, found quickly and audited.

Steps to create a custom object

1. From the setup page, click **Object Manager**.
2. Click **Create** and then **Custom Object**.
3. Enter the label name, plural label name and record name (see the table below).
4. Select the data type of the record name as **Text**.
5. Select **Allow reports**, **Allow search** and **Track Field History**.
6. Click **Save** (or **Save & New** to create the next object).

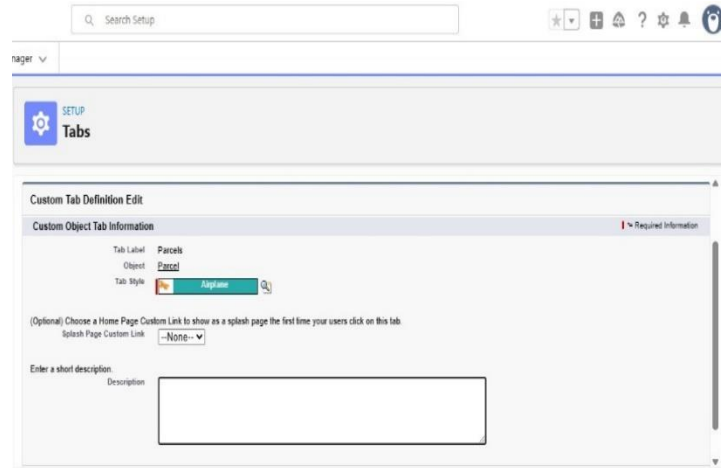
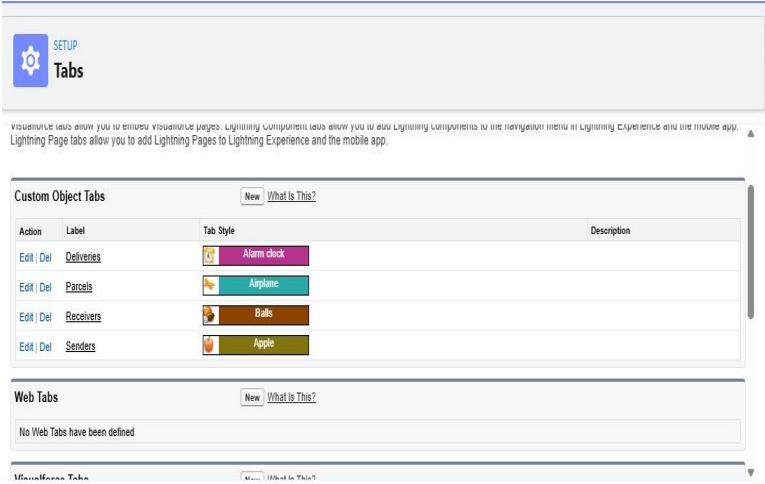
Object	Label	Plural label	Record name	Record name type	API name
Parcel	Parcel	Parcels	Parcel Name	Text	Parcel__c
Delivery	Delivery	Deliveries	DeliveryName	Text	Delivery__c
Sender	Sender	Sender's	SenderName	Text	Sender__c
Receiver	Receiver	Receiver's	ReceiverName	Text	Receiver__c

Milestone 3 – Tabs

A tab lets users open an object from the navigation bar. We created a custom object tab for each of the four objects so that staff can reach Parcels, Deliveries, Senders and Receivers directly.

Creating a tab for the Parcel object

1. Go to the setup page and type **Tabs** in the Quick Find bar.
2. Click **Tabs**.
3. Click **New** under Custom Object Tabs.
4. Select the object (Parcel) and select a tab style.
5. Click **Next**, keep the Add to Profiles settings as default and click **Next** again.
6. On the Add to Custom Apps page, uncheck **Include Tab**.
7. Make sure **Append tab to the user's existing personal customizations** is checked.
8. Click **Save**.



Creating the remaining tabs

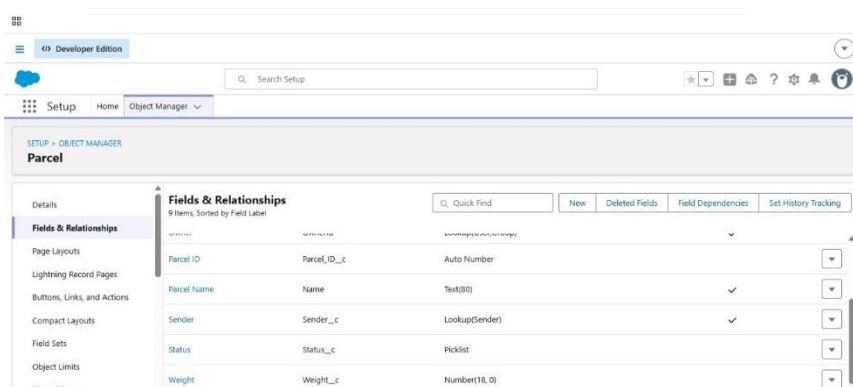
We repeated the same procedure for the Delivery, Sender and Receiver objects, choosing a tab style for each and saving.

Milestone 4 – Fields & Relationships

Fields hold the actual data on each record, and relationships (lookups) connect the objects. We created the fields below for each object. To create a field, open **Object Manager**, select the object, go to **Fields & Relationships**, click **New**, choose the data type, enter the label and details, and click **Next** → **Next** → **Save & New**.

Parcel object – fields

Field name	API name	Data type	Description
Parcel ID	Parcel_ID__c	Auto number (P-{000})	Unique ID for each parcel
Status	Status__c	Picklist	Booked, In Transit, Out for Delivery, Delivered
Weight	Weight__c	Number	Weight of parcel
Estimated Delivery	Estimated_Delivery_Date__c	Date	Expected delivery date
Sender	Sender__c	Lookup relationship	Gets details from Sender

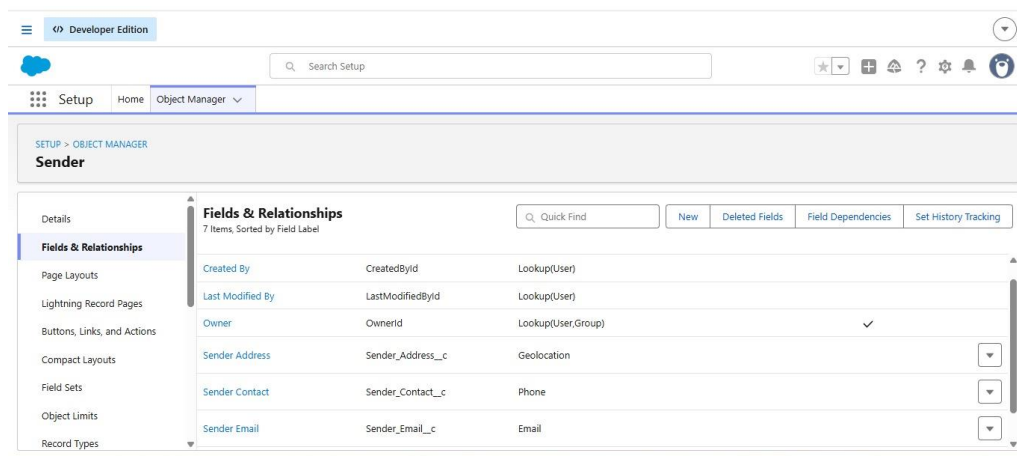


Delivery object – fields

Field name	API name	Data type	Description
Current Location	Current_Location__c	Geolocation	Location in terms of longitude and latitude
Estimated Delivery	Estimated_Delivery_Date__c	Date	Expected delivery date
Sender	Sender__c	Lookup relationship	Gets details from Sender
Parcel	Parcel__c	Lookup relationship	Gets details from Parcel

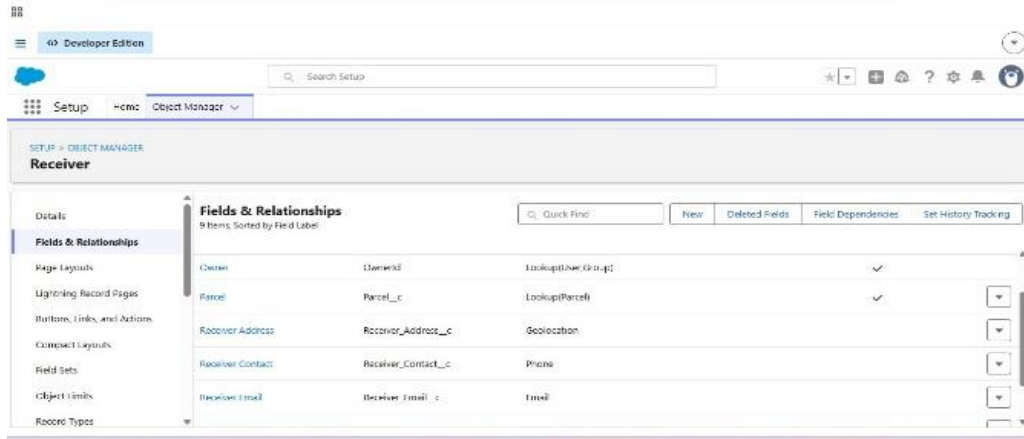
Sender object – fields

Field name	API name	Data type	Description
Sender Address	Sender_Address__c	Geolocation	Location in terms of longitude and latitude
Sender Contact	Sende_Contact__c	Phone	To contact the sender
Sender Email	Sender_Email__c	Email	To send e-mail notifications



Receiver object – fields

Field name	API name	Data type	Description
Receiver Address	Receiver_Address__c	Geolocation	Location in terms of longitude and latitude
Receiver Contact	Receiver_Contact__c	Phone	To contact the receiver
Receiver Email	Receiver_Email__c	Email	To send e-mail notifications
Sender	Sender__c	Lookup relationship	Gets details from Sender
Parcel	Parcel__c	Lookup relationship	Gets details from Parcel



Milestone 5 – Prompt Builder

Prompt Builder is the Einstein generative-AI tool that lets us write a reusable prompt template and ground it with real Salesforce data. We designed a **Record Summary** template that reads a Parcel record and returns a neat tracking update. Later, the Agentforce agent calls this template through a Flow.

Creating the prompt template

1. In Salesforce, go to **Setup**.
2. In the Quick Find box, type **Prompt Builder** (or **Prompt Templates**) and open it.
3. Click **New Prompt Template** and choose the **Record Summary** template type.
4. Select the object type **Parcel (Parcel__c)** and click **Start from Scratch**.
5. Enter the details: Label = **Retrieve Parcel Details**, API Name = **Retrieve_Parcel_Details**, Description = "Retrieves parcel details where Parcel_ID matches the user input."

Entering the prompt template content

In the Prompt Template text area we pasted the following text exactly:

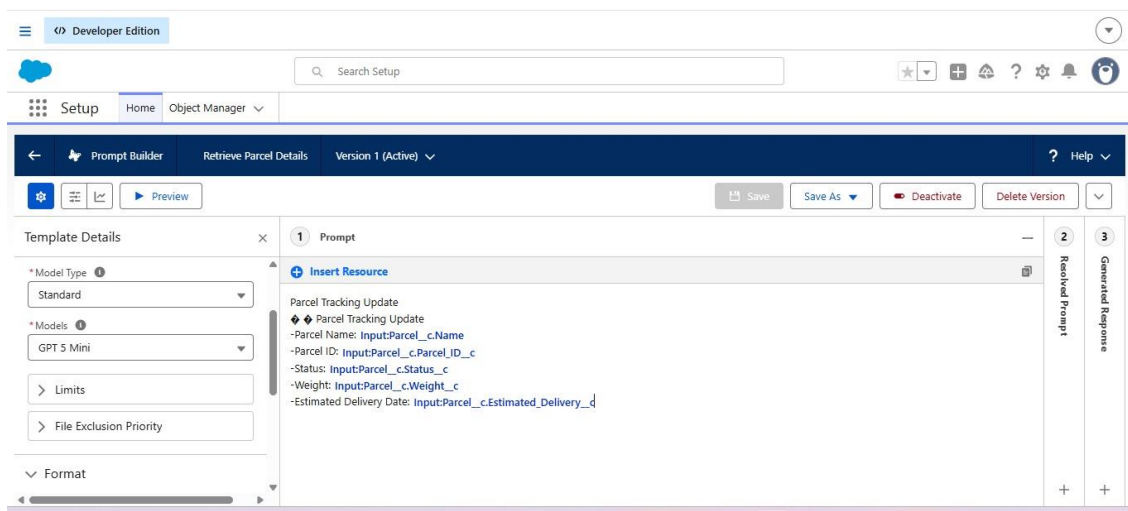
```
Retrieve details from Parcel__c where Parcel_ID__c matches user input.
Return response in this format:
```

```
Parcel Tracking Update
```

```
- Parcel Name: {!$Input:Parcel__c.Name}
- Parcel ID: {!$Input:Parcel__c.Parcel_ID__c}
- Status: {!$Input:Parcel__c.Status__c}
- Weight: {!$Input:Parcel__c.Weight__c}
- Estimated Delivery Date: {!$Input:Parcel__c.Estimated_Delivery_Date__c}
```

Adding resource mapping

1. Scroll down to the **Resources / Insert Resources** section.
2. Click **Insert Resource** and choose **Object**.
3. Select **Parcel__c**.
4. Add the fields Name, Parcel_ID__c, Status__c, Weight__c and Estimated_Delivery_Date__c, clicking Insert Resource (or Add) for each one.
5. Click **Save**.
6. Click **Activate** to make the prompt template available for the agent or flow.



Testing the prompt

To test the prompt we used the preview mode of Prompt Builder: we selected a Parcel record, switched on **Generate Response**, and compared the resolved prompt with the generated response. The generated response listed the parcel name, parcel ID, status, weight and estimated delivery date exactly as defined in the template. From the agent preview, a message such as “Track parcel P-12345” should return the details from the Parcel record.

Milestone 6 – Flows

The Flow is the bridge between the agent and the prompt template. It receives a Parcel ID from the agent, looks up the matching Parcel record, passes it to the Prompt Builder template and hands the generated text back as output. We built it as an **Auto-launched Flow (No Trigger)**.

Creating the flow and its variables

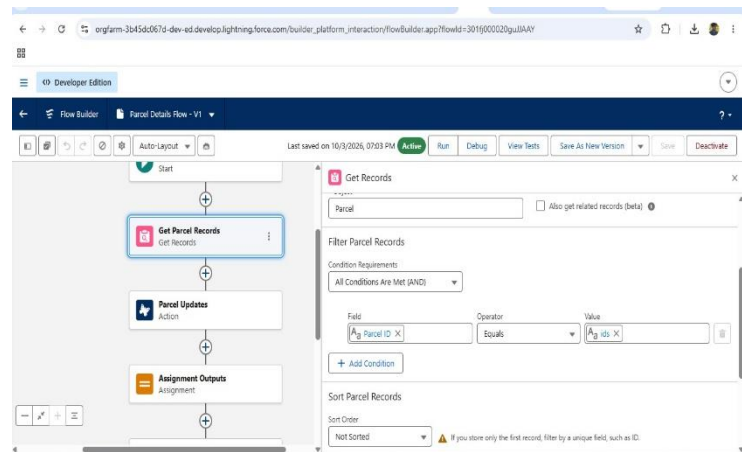
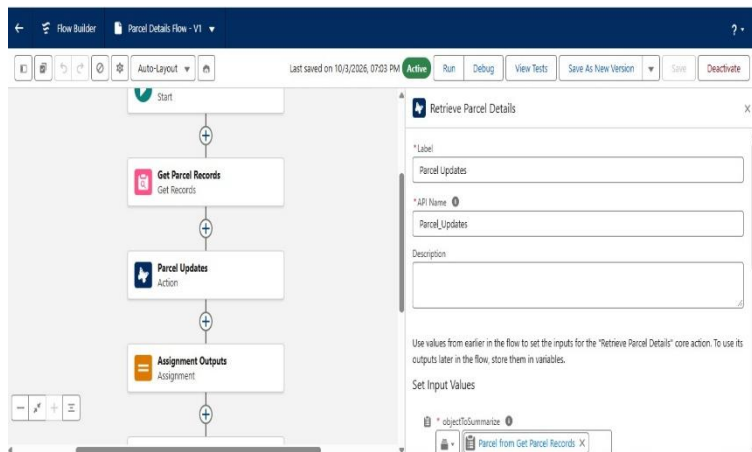
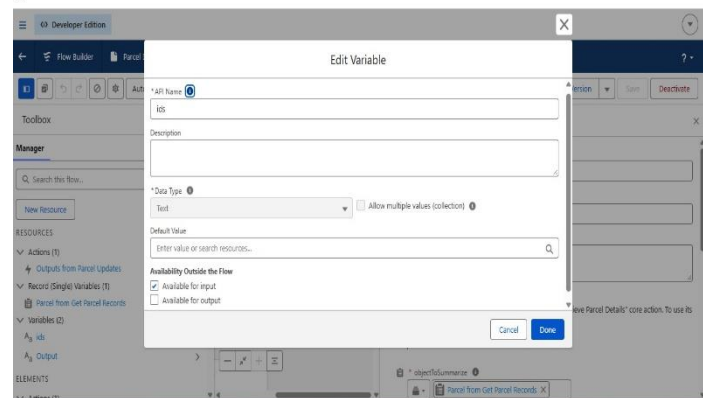
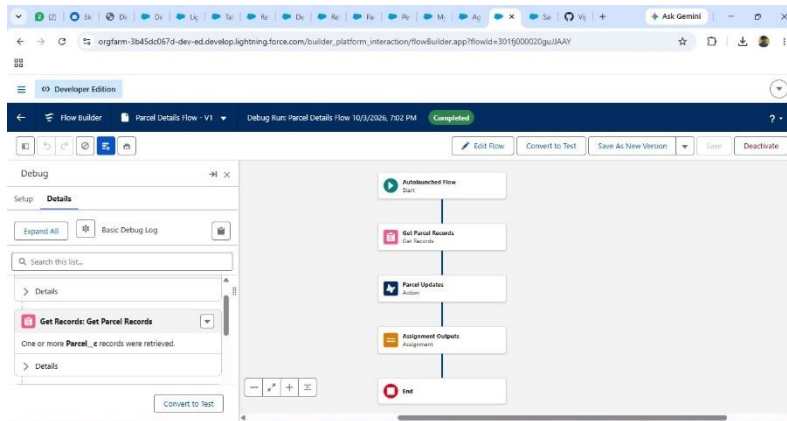
1. Go to Setup, search for **Flows** and click **New Flow**.
2. Choose **Auto-Launched Flow (No Trigger)** and click **Create**.
3. Add the input variables so the agent can pass values: click **New Resource → Variable**.
4. Create a variable named **ids**, data type Text, and tick **Available for input**.
5. Create a variable named **output**, data type Text, and tick **Available for output**.

Adding the flow elements

Order	Element	Label	Configuration
1	Get Records	Get Parcel Records	Object = Parcel; Field = Parcel_ID__c; Operator = Equals; Value = ids
2	Action	Parcel Updates	Find and select the prompt template Retrieve Parcel Details; Object To Summarize = Parcel from Get Parcel Records
3	Assignment	Assignment Outputs	Variable = output; Operator = Equals; Value = Outputs from Action > Prompt Response

Connecting, saving and activating

1. Connect **Start** → **Get Records** → **Action** → **Assignment** → **End**.
2. Click **Save** and name the flow **Parcel Details**.
3. Click **Activate**.
4. Return to the Flows list, open the drop-down next to the new flow and click **Edit Access**.
5. At the top, look for **Flow user access**, assign the available profiles and click **Save**.



Phase 3: UI/UX Development & Customization

This phase enables agents to manage parcel operations efficiently and ensures secure access control by using permission sets, together with AI-driven agent experiences built with Agentforce.

Milestone 1 – Creating Permission Sets

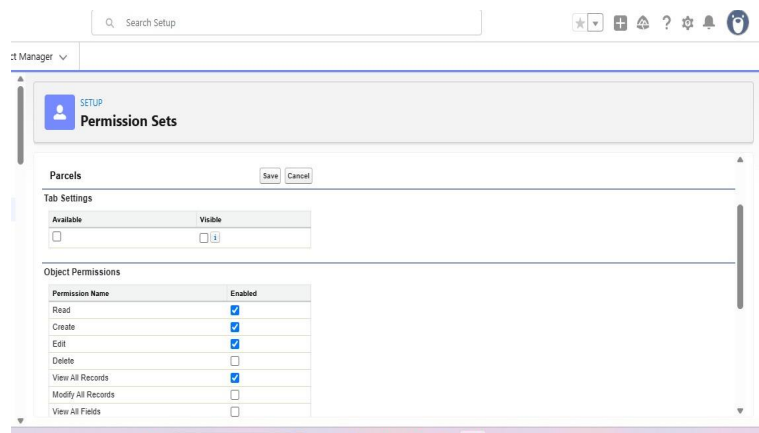
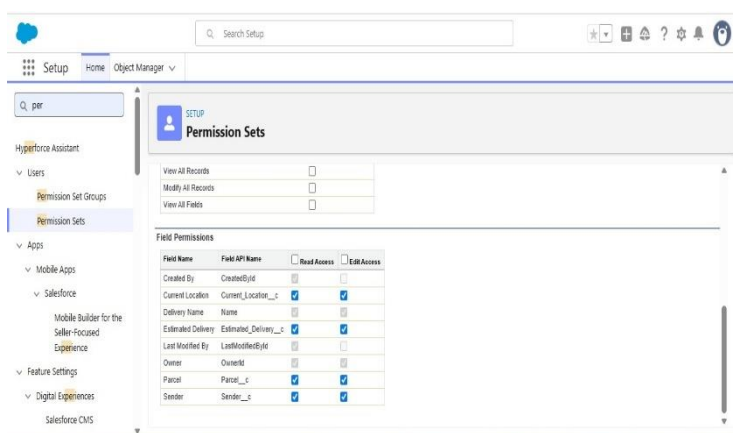
A permission set grants access without changing a user's profile. We created one permission set called **Swift Ship** for the agent user, so the agent can read and write only the SwiftShip objects it needs.

1. In the Quick Find box, type **Permission Sets** and click **Permission Sets** under Users.
2. Click **New**.
3. Enter the details: Label = **Swift Ship**, API Name auto-fills, License = **Einstein Agent**.
4. Click **Save**.
5. Inside the permission set, under **Apps**, click **Object Settings**.
6. Search for and click each object one by one, then click **Edit**.

Access enabled for each object

For every object we enabled **Read**, **Create** and **Edit** (Delete is optional if agents should remove records). Under Field Permissions we ensured that at least Visible is checked for all key fields, and clicked Save after each object.

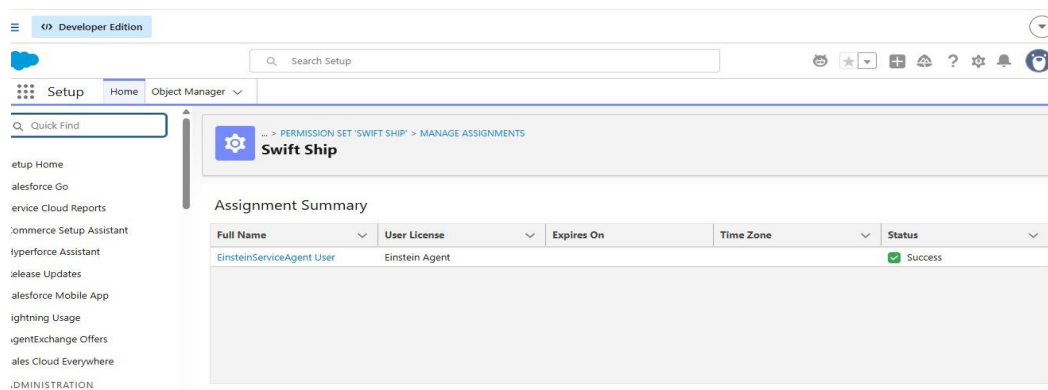
Object	Read	Create	Edit	Delete	Key fields made accessible
Parcel__c	Yes	Yes	Yes	Optional	Parcel ID, Estimated Delivery Date, Sender, Status, Weight
Delivery__c	Yes	Yes	Yes	Optional	Current Location, Estimated Delivery Date, Parcel, Sender
Sender__c	Yes	Yes	Yes	Optional	Sender Address, Sender Contact, Sender Email
Receiver__c	Yes	Yes	Yes	Optional	Parcel, Receiver's Address, Receiver's Contact, Receiver's Email, Sender



Milestone 2 – Assigning Permissions to the Agent User

The permission set only takes effect once it is assigned. Agentforce runs as a special **Einstein Service Agent user**, so that user must hold the Swift Ship permission set; otherwise the flow cannot read Parcel records when the agent calls it.

1. In the Quick Find box, type **Users** and click **Users** under the Users section.
2. From the list of users, find the **Einstein Service Agent User** that represents your agent.
3. Click the full name of that user to open the record.
4. Scroll down to **Permission Set Assignments** and click **Edit Assignments**.
5. On the left (Available Permission Sets), select **Swift Ship**.
6. Click **Add →** to move it to the Enabled Permission Sets list.
7. Click **Save**.



Milestone 3 – Org Setup for Agentforce

Enable the Salesforce default agent from Salesforce Go

1. From the Quick Find box, search for **Salesforce Go** and click it.
2. Search for **Default Agent**, click **Get Started** and turn it on.
3. Refresh the page.

Enable Einstein Setup

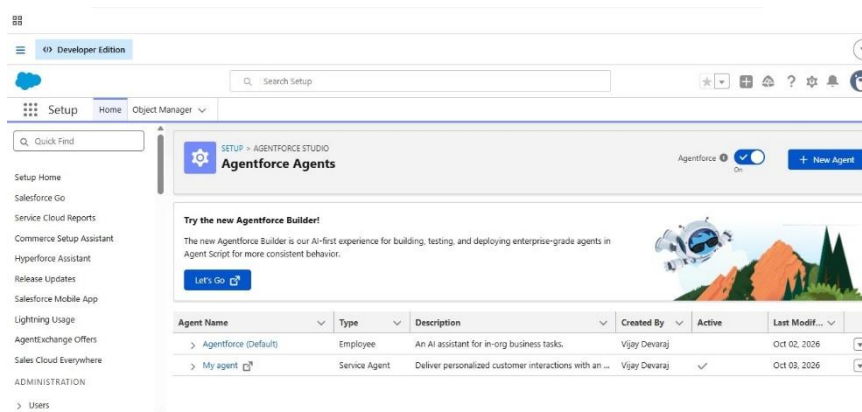
1. Go to Setup → Quick Find → **Einstein Setup** and click **Get Started** if prompted.
2. Toggle on **Turn On Einstein**, **Global Languages** and **Deploy Prompt Templates**.
3. Refresh your browser page.
4. Go to Setup → Quick Find → **Agentforce Agents**.
5. Toggle on **Agentforce** and **Enable Agentforce (Default) Agent**.

Milestone 4 – Building the Agent

With the org prepared, we built the agent itself. We opened the default Agentforce agent in Agentforce Builder, created a subagent dedicated to parcel details, gave it an action that runs our Flow, renamed the agent and activated it.

Open the agent in Builder

1. In Setup, search for **Agentforce Agents** in Quick Find.
2. Click Agentforce Agents, click the default agent and then click **Open in Builder**.



Assign a subagent to the agent

On the Subagents panel, click **New → New Subagent**, and describe what the subagent should do. Ours reads: “The Agent should give the parcel details related to the user, by asking for the parcel Id.” The builder then generated the name **Parcel Details**, a classification description (“Handles user requests to retrieve parcel details by asking for the parcel ID”), a scope (“Your job is only to provide parcel details by obtaining the parcel ID from the user and retrieving the relevant information”) and instructions.

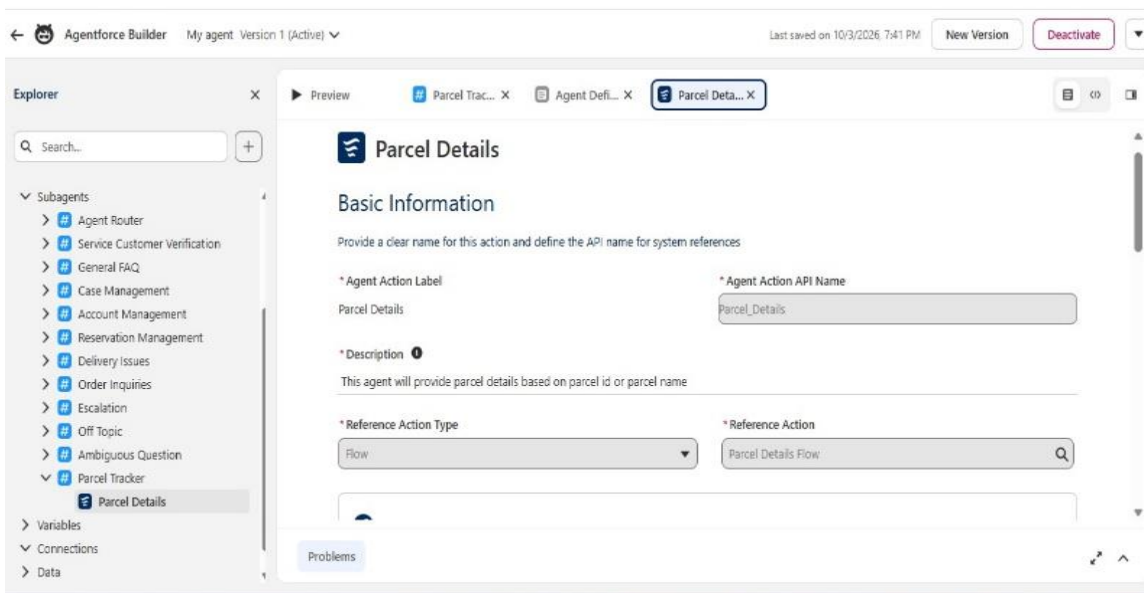
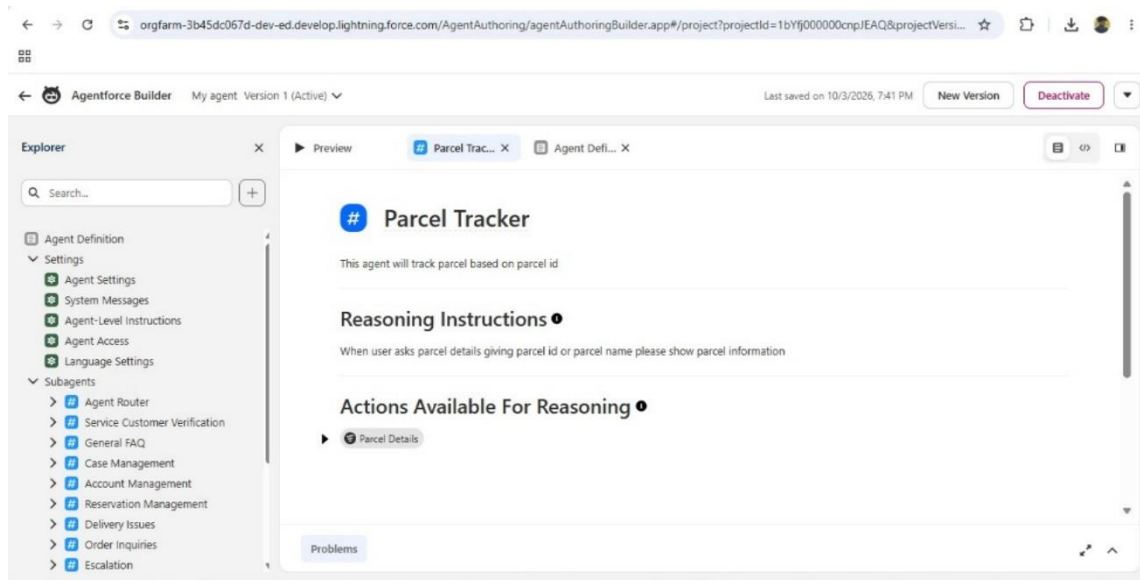
Adding the action to the Agentforce asset library

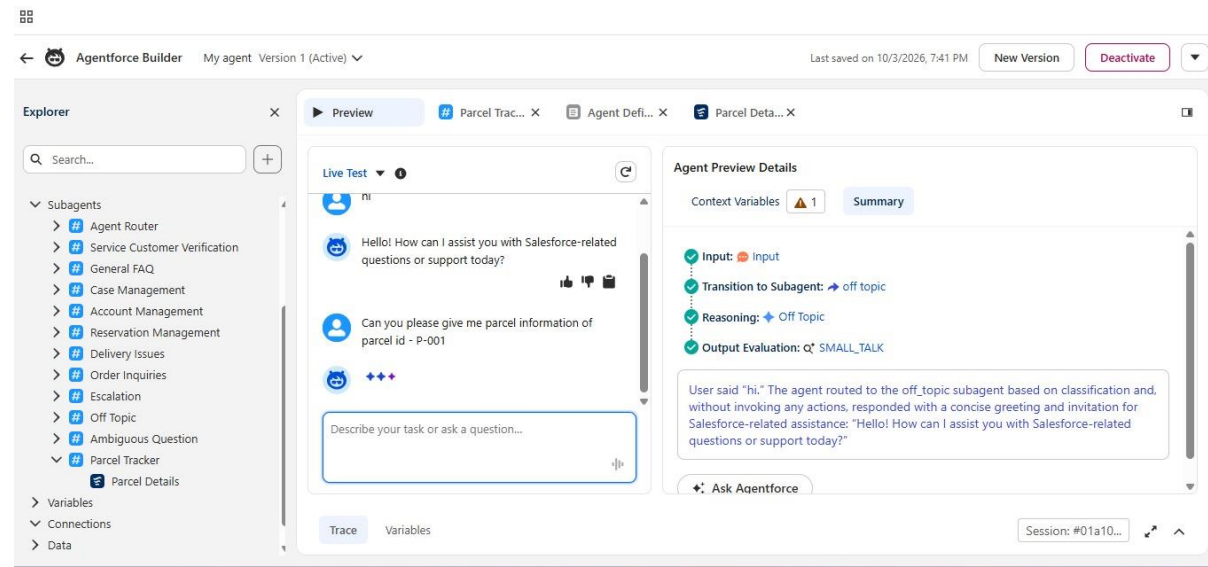
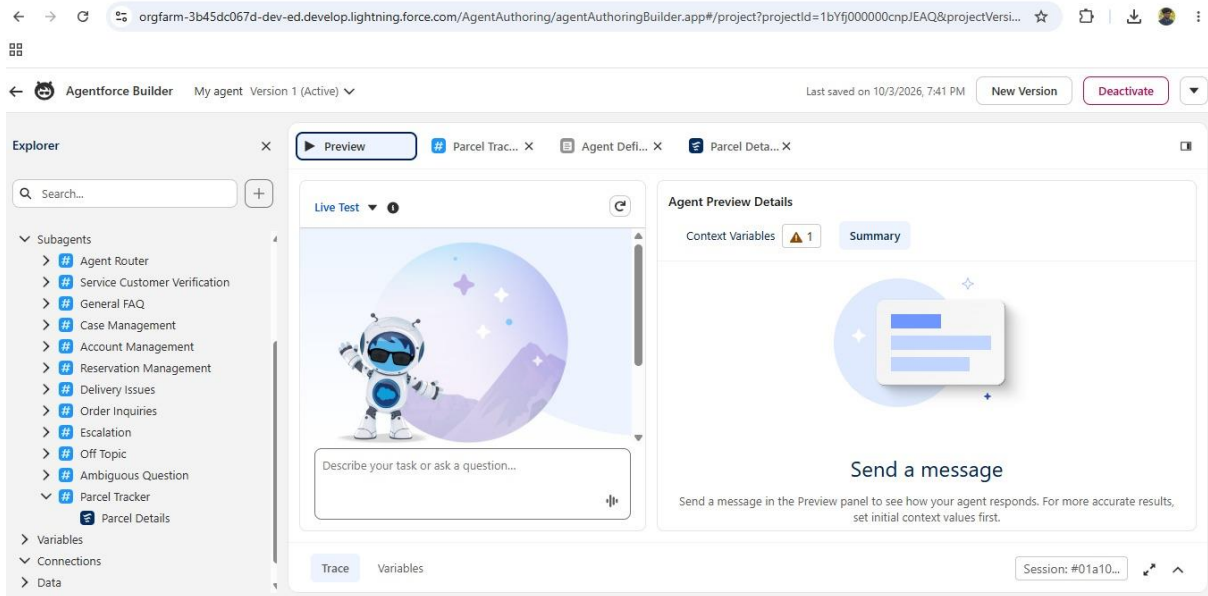
1. Open **Agentforce Assets** (Agentforce Asset Library) and click the **Actions** tab.
2. Click **New Agent Action**.
3. For Reference Action Type choose **Flow** and click **Next**.
4. For Reference Action select the recently created flow (**Parcel Details flow**) and click **Next**.
5. Fill in the details for the agent configuration (action label, instructions and input/output descriptions).

Assign the action to the subagent

1. While in Agentforce Builder, click the **Parcel Details Inquiry** subagent.
2. Open **This Subagent’s Actions**, click **New** and select **Add from Asset Library**.
3. Select the flow action we created (**Parcel Details**) and click **Finish**.

4. Click **Settings** and rename the agent to **My agent**.
5. Click **Save**.
6. Click **Activate**; if a Configuration Issues Detected message appears, click **Ignore & Activate**, then activate.





Phase 4: Data Migration, Testing & Security

This phase validates the functionality and reliability of the agent created in Agent Builder. It ensures that the agent correctly recognizes topics, triggers the assigned Flow, retrieves parcel data and performs all intended actions without access or permission issues. The goal is to confirm that the AI-driven agent operates seamlessly within Salesforce and provides accurate, real-time responses to parcel management queries.

Verify the agent

1. Go to Setup → **Agentforce Agents**.
2. Open the Agents list and confirm that the **My agent** agent appears and is active.

Validate the assigned topic (subagent)

In Agent Builder → Subagent, confirm that the subagent you created (Parcel Details Inquiry) is listed. Open it to verify that it contains a prompt or description related to parcel management and that the Action/Flow is linked. Click Save and ensure the subagent is active.

Check the flow association

1. Go to Setup → **Flows** and locate the Parcel Details flow.
2. Confirm that the flow is **Activated**.
3. Confirm that the flow handles Parcel ID input, parcel details retrieval and the confirmation message (output).
4. Open the flow's Advanced Settings and, if data access is restricted, ensure **Run in System Context (Without Sharing)**.

Activate and test the agent

1. Return to Setup → Agentforce Builder and open the agent.
2. Click **Activate** (if not already activated).
3. Once active, test using the Conversation Preview, for example by typing "Track parcel ID P12345".
4. Observe the agent's response and the reasoning steps (topic selection and action selection).

Validate data in Salesforce

After the agent runs the Flow, open the App Launcher, go to **Parcels (Parcel__c)** and search for the Parcel ID used during testing. Verify that the Status and related fields are correct and that records created through the Flow appear correctly. We also opened a Receiver record (Receiver, Sender and Parcel lookups) to check that the relationships display the linked records.

Phase 5: Deployment, Documentation & Maintenance

Deployment scope

The purpose of this phase is to validate that the SwiftShip Tracker agent, created using Salesforce Agent Builder, works as intended within the Salesforce Developer Org environment. The main goal is to understand how an AI-powered service agent can be tested to ensure correct topic recognition, Flow execution and data retrieval from Salesforce custom objects.

Testing was conducted within the Developer Org to simulate how the agent would perform in a live setup, ensuring that permissions, flows and interactions function correctly before any deployment stage. Real-world production testing (UAT or sandbox deployment) was **not** performed in this project because:

- Developer Edition orgs are standalone environments and do not support production-linked testing environments.
- Enterprise-level Agentforce deployment involves sandbox testing, staging environments and CI/CD automation, which are outside the scope of this academic/prototype implementation.

Maintenance, monitoring and troubleshooting

In the SwiftShip Tracker project, ongoing maintenance ensures that Parcel, Customer and Delivery data remain accurate, synchronized and up to date. Regular monitoring helps detect issues such as misrouted agent queries, failed Flow executions or performance delays in Agentforce responses. Troubleshooting activities include resolving configuration errors, updating Flow logic, refining agent topics and improving prompts to enhance user interactions.

Although this project is implemented within a Salesforce Developer Org for learning purposes, in real-world enterprise deployments these maintenance and monitoring practices are essential. They ensure that AI-powered service agents remain reliable, scalable and continuously aligned with evolving customer service operations and business goals.

Symptom	Likely cause	What to check
Agent says it cannot find the parcel	Wrong or missing Parcel ID; field not visible	Search the Parcel record; check field-level visibility in the Swift Ship permission set.
Agent does not pick the parcel subagent	Classification description or scope too vague	Refine the subagent description and instructions in Agentforce Builder.
Flow fails when called by the agent	Flow not active or no user access	Confirm the flow is active; review Flow user access and run context.
Prompt returns blank values	Resource mapping missing	Open Prompt Builder, re-insert the Parcel__c field resources and preview again.
Agent cannot activate	Configuration issues	Review the Activation Checklist, then fix or ignore warnings as appropriate.

7. Functional & Performance Testing

During testing, we captured screenshots from Salesforce to validate each feature of SwiftShip Tracker: object and record creation, field relationships, the Prompt Builder preview, the Flow, permission-set assignment and the Agentforce conversation. All features were verified to ensure correct functionality and performance.

7.1 Functional test cases

ID	Test scenario	Expected result	Status
TC-01	Create a Parcel record	Record saves with auto-numbered Parcel ID (P-001 format)	Pass
TC-02	Create Sender and Receiver records and link them to a Parcel	Lookups display the linked Sender and Parcel	Pass
TC-03	Create a Delivery record with current location	Geolocation values are stored against the Parcel	Pass
TC-04	Preview the Retrieve Parcel Details prompt for a Parcel record	Resolved prompt and response show name, ID, status, weight and estimated delivery date	Pass
TC-05	Run the Parcel Details flow with a valid Parcel ID	Output variable contains the prompt response	Pass
TC-06	Ask the agent “can I get my parcel details”	Agent asks for the Parcel ID	Pass
TC-07	Reply with a valid Parcel ID (for example P-123)	Agent returns the Parcel Tracking Update with correct details	Pass
TC-08	Reply with an unknown Parcel ID	Agent does not return another parcel’s data	Pass
TC-09	Remove the Swift Ship permission set from the agent user (negative test)	Agent cannot retrieve the data	Pass

7.2 Performance observations

Performance for this project was judged on how quickly the agent understood and acted on a request. In the Agentforce reasoning plan for the query “sure my parcel ID is P-123”, the topic-selection step took about 0.22 seconds and the action-selection step about 0.38 seconds, and the complete parcel summary was returned in the same conversation turn. For a Developer Edition org with a handful of test records this is well within what a customer would consider instant.

Metric	Observation
Topic selection (subagent routing)	≈ 0.22 seconds
Action selection (flow call)	≈ 0.38 seconds
Response content	Parcel name, ID, status, weight and estimated delivery date
Test data volume	Small (Developer Edition test records)

8. Advantages & Disadvantages

8.1 Advantages

- Centralized parcel and customer information in Salesforce.
- Provides better visibility into parcel booking and delivery tracking.
- Automates key logistics processes using Salesforce Flow.
- Agentforce enables conversational customer support and parcel-status assistance.
- Prompt Builder supports AI-powered responses and self-service.
- Reduces manual effort for customers, support teams and administrators.
- Provides timely notifications and updates during the delivery process.
- Experience Cloud can provide customers with convenient access to tracking information.
- Supports integration with external logistics systems.
- Can scale to support multiple locations and growing parcel operations.

8.2 Disadvantages

- Requires accurate and up-to-date parcel and customer data.
- Dependence on Salesforce configuration and automation.
- Agentforce responses depend on properly configured prompts and available data.
- External logistics integrations may increase implementation complexity.
- Flow and automation logic require regular maintenance when business processes change.
- Users need appropriate Salesforce or Experience Cloud access to use the system.
- Initial Salesforce implementation and configuration can require significant effort.
- Expanding the system to additional locations or integrations may require further customization.

8.3 Our assessment

On balance, the advantages outweigh the disadvantages for a growing parcel business. Most of the drawbacks are about discipline rather than technology: keeping data clean, maintaining prompts and flows, and managing access. These are manageable with the monitoring and troubleshooting practices described in Phase 5.

9. Conclusion

The SwiftShip Tracker demonstrates how Salesforce can be used to build a centralized parcel management and tracking solution. The project integrates custom objects, relationships, Flows, Apex, Email Alerts, Experience Cloud, Agentforce AI, Prompt Builder, security controls and reporting to support parcel booking, delivery tracking, status updates and customer communication.

The integration of Agentforce AI with Prompt Builder and Salesforce Flow provides a conversational approach to parcel tracking, enabling customers to obtain shipment information without depending

entirely on manual support. The project therefore provides a practical foundation for a scalable and user-friendly parcel management solution.

As a team, this project taught us how a business problem is taken from an empathy map to a working system: how design-thinking tools shape requirements, how an Agile backlog keeps work organised, and how Salesforce’s declarative tools can be combined with generative AI. It also showed us the importance of permissions and data quality, since the smartest agent is only as good as the data and access it is given.

10. Future Scope

SwiftShip Tracker is a working prototype with a clear path for growth. The areas we identified for future enhancement are enterprise deployment, continuous monitoring, prompt refinement, Flow improvement and AI-agent scalability.

Future enhancement	What it would add
Extend Agentforce capabilities	Handle more parcel-management queries, such as booking a parcel or updating a delivery address through conversation.
Enhance real-time delivery tracking	Show the parcel’s current location on a map using the Delivery geolocation data.
Expand customer self-service through Experience Cloud	A branded portal where customers book, track and manage parcels.
Add advanced delivery-performance analytics	Dashboards for on-time delivery rate, agent productivity and route performance.
Integrate additional external logistics systems	Carrier and partner APIs for shipment updates and delivery status.
Improve AI prompts and conversational experiences	More natural, multilingual and context-aware responses.
Introduce enterprise-level sandbox and CI/CD deployment	Safe promotion of changes from sandbox to production with automated testing.
Expand automation for additional parcel operations	Automatic notifications, escalations and scheduled status checks.

11. Appendix

11.1 Components used

Ref	Category	Components
A	Custom Objects	Parcel, Delivery, Sender, Receiver
B	Standard Objects	Account, Contact, Case, Task, EmailMessage, User
C	Automation	Flows, Approval Processes, Email Alerts
D	Apex	Apex Classes, Batch Apex
E	AI	Agentforce AI, Prompt Builder
F	UI	Lightning App, Lightning Record Pages, Experience Cloud
G	Security	Profiles, Roles, Permission Sets, Sharing Rules, Field-Level Security
H	Analytics	Reports & Dashboards
I	Testing	Flow, Agentforce, data retrieval, security and permission validation

11.2 Glossary

Term	Meaning
Agentforce	Salesforce's platform for building AI agents that can take actions on behalf of users.
Prompt Builder	Tool for creating reusable prompt templates grounded in Salesforce data.
Flow	Declarative automation built in Flow Builder.
Subagent (topic)	A defined area of responsibility for an agent, with its own scope, instructions and actions.
Custom object	A table created in Salesforce to store data specific to a business, with names ending in __c.
Lookup relationship	A field that links one record to another record.
Permission set	A collection of permissions that extends a user's access.
Developer Edition	A free Salesforce org used for learning and development.
Story point	A relative unit used in Agile to estimate the effort of a user story.
Velocity	The number of story points a team completes in a sprint or period.

Thank You